# Introduction to Python

**Python:** It is a General purpose High level programming language.

→ Let's break that down and understand each word in a simple way:

**What is programming language?**

It is just a way to communicate with the computer.

These lang have

- Rules (syntax)
- words (keywords & stmts)

→ A programming lang is a set of instructions we give to the computer to make it do what we want

**Types of programming lang**

| Language type | who understands it easily? | characteristics |
|---|---|---|
| High level lang | Humans | — Easy to read & write<br>— looks like english<br>— used to build s/w easily<br>ex: Java, C++ |
| Middle level lang | b/w Human & Machine | — Bit harder to read<br>— uses symbols & machine friendly words<br>ex: C |
| Low level lang | Machine | — Fully in binary (1's & 0's)<br>— Hard for humans to read<br>ex: Machine code, lang |

## Who created python?

Name : Guido van Rossum

Country : Netherlands (Dutch programmer)

Year : 1991

→ He wanted to create a lang that is easy, powerful & fun

## Current python version info

Most popular version — python 3

Latest version — python 3.14.2

Who Maintains — python s/w Foundation.

## Features of python:

1) Easy to learn & Analyze
2) Interpreted lang
3) Dynamically Typed lang
   - You don't need to define the datatype of a variable
   - python figures out the datatype at runtime
4) Open source
5) Huge library support (7+crore functions)
6) Built in Garbage collector
   - Automatically removes unused memory / data.
   - You don't need to manually delete things.
   - keeps your prgm lightweight & optimized.

## Tokens:

→ A token is the smallest individual unit in a python. prgm.

→ All stmts & instructions in a prgm are built with tokens.

The various tokens in python are:

1) Identifier
2) keyword
3) Literal
4) operator

## keywords:

→ It are reserved words with predefined meaning
→ they are universal, same in every python prgm
→ you can use them but not modify them

1) Hard keywords
2) soft keywords

**To view keyword:**

```
import keyword
print(keyword.kwlist)
```

Total keywords: 35

Special keywords: True, False, None

**soft keywords**

```
import keyword
print(keyword.softkwlist)
```

Total keywords: 4 (_, match, case, type)

| keyword | Meaning | usage |
|---|---|---|
| True | Boolean (1) (Y\|ON\|True) | used in logical conditions |
| False | Boolean (0) (N\|OFF\|False) | used in logical conditions |
| None | Represents nothing/ null | used to show absence of value |

## Literals

→ It is nothing but values or data.
→ It is used to perform specific task in the prgm
→ Literals are classified into two types
  1) Number literals
  2) Character literals

**Number literals:** Binary, Decimal (Int And float)

**Character literals:** uppercase, lowercase, special character

## Roadmap to slw Development :

Software (collection of application)
↑
Application ( " " library)
↑
Library ( " " package)
↑
package ( " " Module)
↑
Module (A file save with .py)
↑
program (set of instructions
↑
Tokens

## Variable :

→ Variable in python is a container to store value.
→ It is a block where the memory address of my value is stored
→ Variable is a name given to memory location / address where the value is stored.
→ whenever the variable is created there will be a memory allocation will be created inside RAM.
→ The memory allocation will be like,

- Variable space / stack space
- Value space / Heap space

-5 to +256 ⇒ 32 bits
-∝ to -6 (or) +257 to +∝ ⇒ 64 bits

Syntax : var_name = value

Ex: a = 30;
↓ → value
var_name

→ var_name will be in the variable space in stack
→ value will be Heap space

Memory :

| single variable creation : | stack space / variable space | value space / Heap space |
|---|---|---|
| | variable name = a | value = 30 |
| | address (0x11) | address (0x11) |

How to get the actual address of the variable?
— By using id() method we can get actual address of the variable and also for the value
— It will give integer address

Syntax: id (var_name / value)

Multi variable creation :

→ python allows you to assign values to multiple variables in one line

Syntax: var1, var2, - - - - varn = val1, val2, - - - valn

Ex: a, b, c, d = 10, 20, 30, 40

```
print(a)
print(b)
print(c)
print(d)
```

Too many values to unpack;
— If we are giving some number variable name but we are giving some number of values we will get error.

Ex: a,b,c = 10,20,30,40

```
print(a)
print(b)
print(c)
print(d)
```

O/P: Error

Not enough values to unpack

Ex: a,b,c,d = 10,20,30

```
print(a)
print(b)
print(c)
print(d)
```

O/P: Error

Rule to create Multiple Variable creation:

No. of Variables == No. of values

Memory allocation multiple Variable creation:

| Variable space / stack space | Value space / Heap space |
|---|---|
| variable name = a<br>add (0x11) | Value = 10<br>add (0x11) |
| variable name = b<br>add (0x12) | Value = 20<br>add (0x12) |
| variable name = c<br>add (0x13) | Value = 30<br>add (0x13) |
| variable name = d<br>add (0x14) | Value = 40<br>add (0x14) |

How memory Allocation for same value with different variables works?

→ when we are trying to store number from -5 to 256 it wont create one more duplicates block it will share the same memory address for the variables which having the same values.

→ once the reference count gets 0 the value get removed from the heap space / value space and collected by garbage collector.

Ex: a,e,i,o,u = 11,12,11,12,13

```
print(a)
print(e)
print(i)
print(o)
print(u)
```

O/P:

```
11
12
11
12
13
```

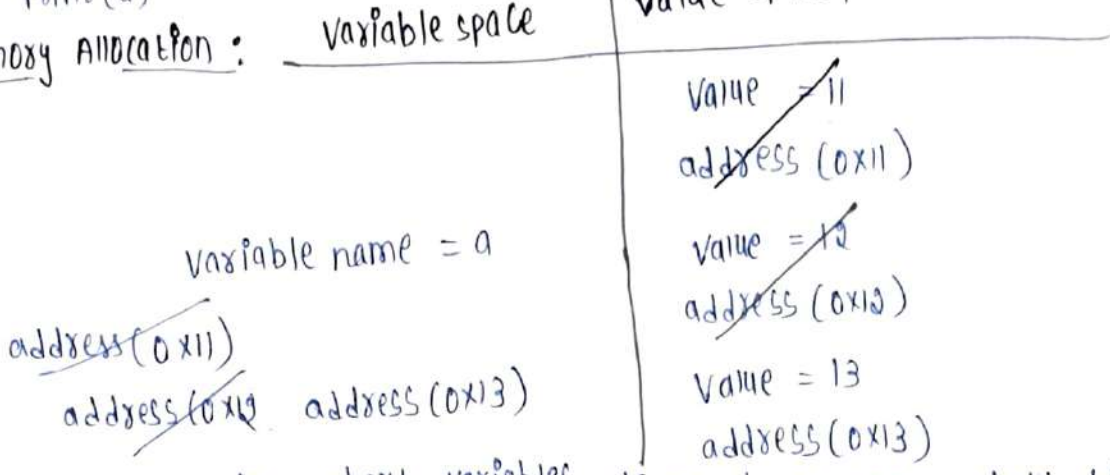
Same variables with diff value how it will allocate memory?

If we have same variable with different value the address of the value will change for each time.

→ when the initialization is done.

Ex: a = 11
a = 19
a = 13
print(a)

Memory Allocation:




When we are talking about variables we need to know about the identifiers.

## Identifier:

— It is a name given to variables, functions, class module (or) other objects.

— All variables are identifiers but all identifiers are not variables.

## Identifier rules:

- Identifier should not be a keyword
- Identifier should not starts with numbers
  2a = 10 ✗
  a2 = 10 ✓
  a2b = 10 ✓
- Identifier should not contain special character except '_' underscore
- Identifier should not contain space in between (or) at the beginning
- Identifier can be alphabets (or) alpha numeric (or) _
- Identifier characters should not cross more than 79 characters. (It is a Industrial standard)

# Datatypes:

- A datatype tells us what kind of data a variable can store.
- Think of it like a Label on a container
  - If the label says "water", you'll know it holds liquid.
  - In same way, datatypes tell Python what kind of data is stored in a variable — numbers, text, decimal values, etc..

## Why Do we need Data types?

Imagine trying to add two things:

- 2 + 3 → makes sense ✓
- 2 + "apple" → Python will get confused ✗

That's why we need data types:

- ✓ To tell Python what kind of value we're working with
- ✓ To perform operations correctly (ex: add, multiply, compare)
- ✓ To store values efficiently in memory
- ✓ Without datatypes, no operation (even simple ones like addition) would be possible!

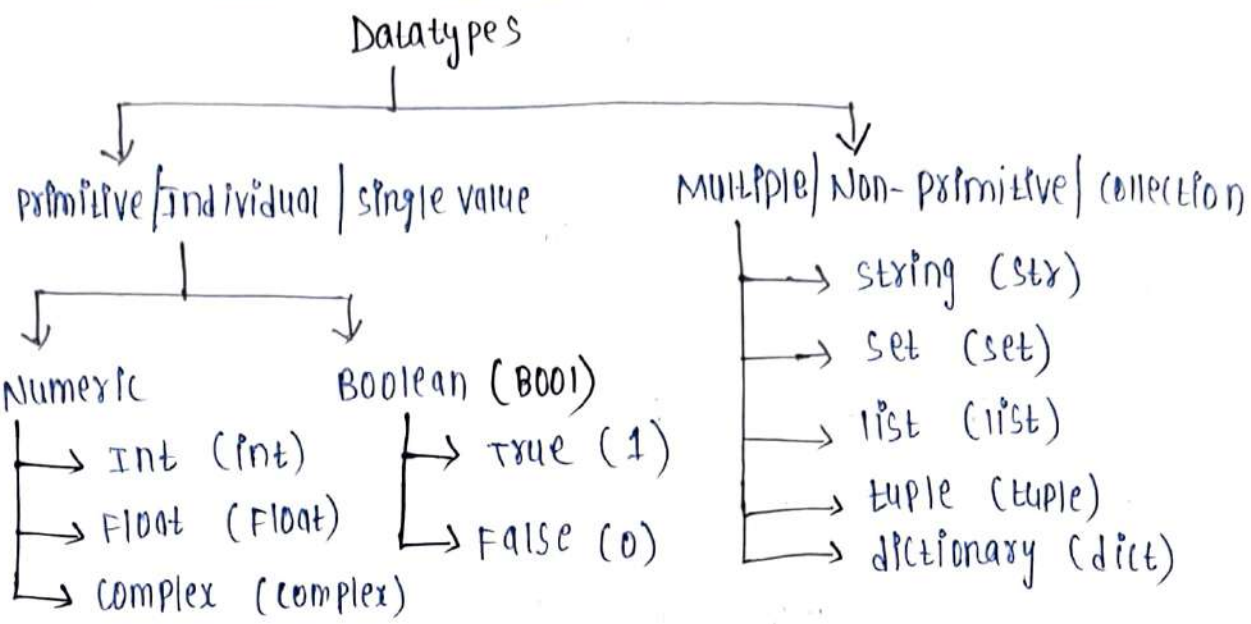
## Classification of Datatypes:

- In Python, datatypes are classified based on the size & structure of data
  1) single value datatype (or) primitive (or) individual — can hold only one value at a time
  2) Multiple value datatype — can hold a group of values at a time
     ↓ (or) Non primitive / collection

## Special concept: Default Vs Non Default Values

Default value — Automatically assigned when there's no meaningful data
ex: (0, False, '')

Non-Default value — Any other value that carries meaning
ex: (2, 3.5, "hello", True)




→ In python :

Default values = considered False

Non Default values = " True

## Single value typed :

This type can store only one value at a time

1) Numeric < Int, Float, Complex

2) Boolean < T, F

### Integer : (int)

— stores real numbers — (no decimal points)

— can be +ve / -ve

— range from $-\infty$ to $+\infty$

Ex : a = 20
b = -100
c = 0

D.value = 0

Non Def.value = other than 0 Ex: 5, -10, 99

## Checking Datatype of a variable :

Python gives us a built in function type() to check what datatype of a value (or) variable is.

Syntax: type (variable - or - value)

Ex: a = 20
print (type(a))

O/P: <class 'int'>

Boolean: Python provides another function called BOOL() to check whether a value is considered True / False

Syntax: bool (vari_or_value)

Ex: a = 20
print (bool(a))

O/P: True

How type() & bool() work Internally?

type() — checks datatype — looks up the object's class/type in memory

bool() — converts value to boolean — returns False for default/empty values (0, '', [], etc) and True for others

## Float: (float)

- A float data type stores decimal numbers
- It must have a decimal point (ex: 2.0, -3.14)
- can be +ve / -ve
- Range: $-\infty$ to $+\infty$
- The digits after the decimal point are called floating values.

Rules:

- You must have atleast one digit before or after the decimal
  0.1, .05, 10.0 ✓
  (.) just dot ✗
- Python allows up to 16 digits precision by default, but it can store more based on system memory

Default value = 0.0
Non default value = other than

64 bits memory

Ex: a = 0.1
b = 0.0
print(type(a)) # <class 'float'>
print(bool(a)) # True

Additional float facts:

— You can write float values using scientific notation

Ex: 1.5e2 means $1.5 \times 10^2 = 150.0$

$3.2e^{-1}$ means $3.2 \times 10^{-1} = 0.32$

— python internally uses IEEE 754 double-precision format to represent float values.

— the float numbers are stored using 64 bits (8 bytes)

— Floating point arithmetic may lead to precision error

Ex: print(0.1+0.2) # o/p : 0.30000000000004

because computers use binary internally and some decimal numbers can't be stored exactly in binary.

## complex :-

— A complex number is made up of

- A real part → any number without 'j'
- An imaginary part → number multiplied by 'j'

syntax: a+bj  $j = \sqrt{-1}$

Ex: a = 3+4j
b = -2-5j

Default value = 0j (or) 0+0j considered False

Non default = other than 0j (or) 0+0j considered True

Ex: a = 0j  print(bool(a)) # False
b = 0+0j  print(bool(b)) # False

64 bits memory

Ex: a = 3j + 2 — Reorder in internally
print(a) # (2+3j)

Ex: z = 5+2j
print(z.real) # o/p: 5.0
print(z.imag) # o/p: 2.0

→ Even if the real (or) imaginary part is given as an integer, Python internally stores and returns them as float values. Because python represents complex numbers as two floating point numbers (double precision) — this helps maintain mathematical precision.

## ▼ Boolean : (bool)

— A boolean datatype stores one of two values: True, False
— used to represent logical decisions in programming.
— True and False are keywords and values in python.
— commonly used in if, while, other control flow stmts.

True — 1
False — 0

32 bits Memory

**Rules:**

— True & False are Reserved keywords
— You cannot assign values to them
— Trying to assign values will raise syntax error
Ex: True = 5 # Invalid

Default value = False
NON d.V = True

Ex: x = True
print(x) # True
print(7==9) # False
a = True
print(type(a)) # shows type

a = True
print(bool(a))
# True

## Multivalue Datatype :

A Multivalue datatype also known as collection datatype, is a data type that allows storing more than one value inside a single variable.

### 1) String : (Str)

- A string is a collection of characters
- Enclosed with quotes (`' '`, `" "`, `''' '''`)
- Characters can include
  - letters : A-Z, a-z
  - digits : 0-9
  - special symbols : @, #, $, etc
  - whitespace characters : space, tab, newline
- Strings are used to represent textual data in python

**Syntax to create :** three ways

i) `s1 = 'Hello Python'`

ii) `s2 = "Hello Python"`

iii) `s3 = '''Hello
This is a multiline
string'''`

With special character = 64 bits
Without " " = 32 bits

**Rules :**

- You must start and end the string with the same type of quotes.
  - `'abc', "abc", '''abc'''` ✓
  - `'abc"` ✗
- Strings are indexed sequences
- It is ordered Datatype
- Strings are immutable (cannot modify after creation)

Default value = empty string `''` consider False

Nond.V = Any non empty string

Indexing : There are two ways to access

1) +ve Indexing (left to right)
— starts from 0

2) -ve Indexing (Right to left)
— starts from 1

Ex:

| Character | h | e | l | l | o |
|---|---|---|---|---|---|
| +ve index | 0 | 1 | 2 | 3 | 4 |
| -ve index | -5 | -4 | -3 | -2 | -1 |

```
s = 'hello'
print(s[0]) # h
print(s[4]) # o
print(s[-1]) # o
print(s[-5]) # h
```

Memory structure :

For s = 'hello', python stores the string as :

index : 0 1 2 3 4

Character: 'h' 'e' 'l' 'l' 'o'

Each character is stored in a separate memory block with an index.

Mutability :

— Strings are immutable — you cannot change a character once the string is created.

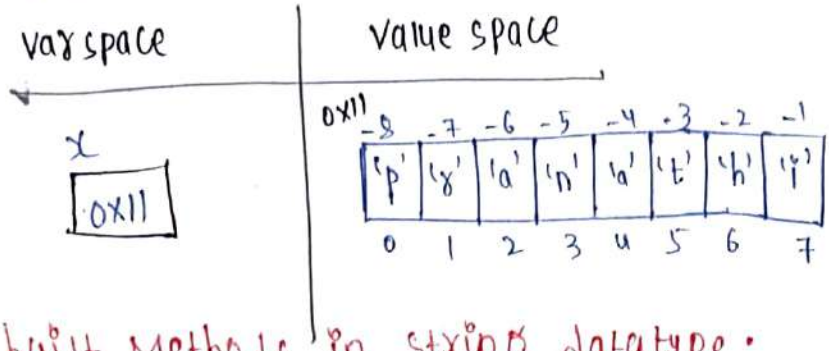
Ex: s = 'hello'

s[0] = 'H' # Error ✗

→ To Modify, create a new string instead

```
a = 'Hello'
b = ' '
print(type(a)) # <class 'str'>
print(bool(b)) # False
```

Ex: x='pranathi'



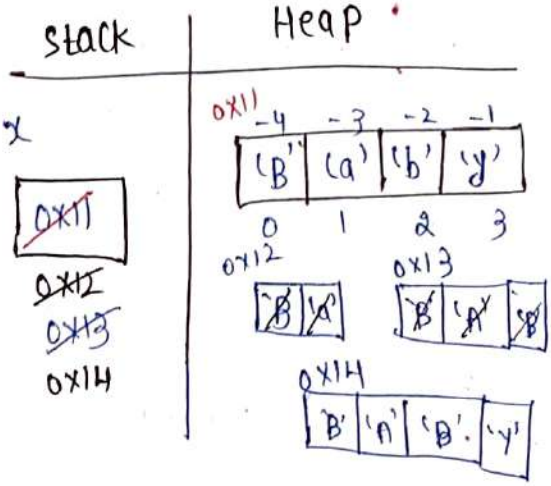

## In built Methods in string datatype:

1) upper()
2) lower()
3) strip()
4) l.strip()
5) r.strip()
6) replace()
7) count()
8) len()
9) swapcase()

### (1) Upper() → converts lower to upper charater

Syntax: `Varname.upper()`

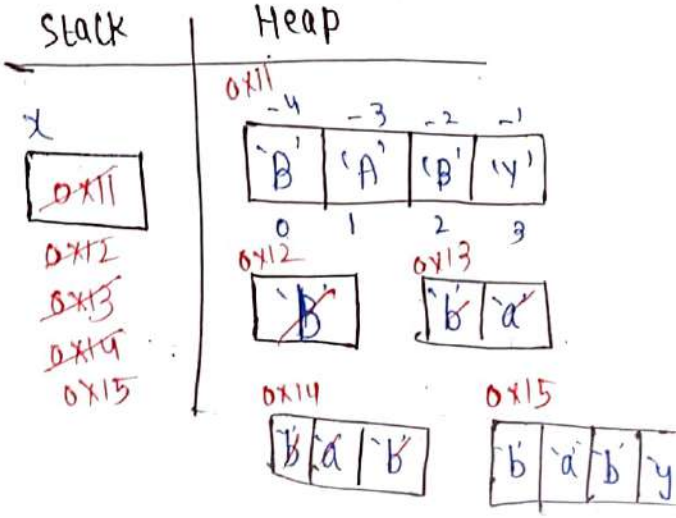
Ex: x = 'Baby'
x.upper() # BABY
x # Baby
x = x.upper() # BABY




### (2) Lower() → converts upper to lower charaters

Syntax: `Varname.lower()`

Ex: x = BABY
x.lower() # baby



3) **strip()**: It will remove the mentioned char at start & end

syntax : `varname.strip('char')`

ex: a=' World '
a.strip(' ') # World

ex: a = '@world@'
a.strip('@') # world

4) **l.strip()**: It remove left side char.

syntax : `varname.lstrip()`

ex: a = '@@world@'
a.lstrip() # world@

5) **r.strip()**: It remove right side char

syntax : `varname.rstrip()`

ex: a = '@@world@'
a.rstrip() # @@world

6) **Replace()**: Replace the sub string (or) char from a string

syntax : `varname.replace('old char', 'new char', no.of times)`

ex: a = 'banana'
a.replace('a', 'o') # bonono
a.replace('a', 'o', 2) # bonona
a.replace('na', 'no') # banono

b = 'banana is yellow'
b.replace('yellow', 'Green') # banana is Green

7) **count()**: count the char in substring

syntax : `varname.count('char', start, stop)`

Ex: a = banana
a.count('a') # 3
a.count('a', 2, 3) # 1

8) len(): It is not a inbuilt func, it is utility fun

Ex: a = 'apple'
len(a) # 5

(9) swapcase(): converting lower → upper and upper → lower

Ex: a = 'HELlO'
a.swapcase() # hEILO

Slicing: Extracting group of characters from a string

Syntax: varname[start index : End index ± 1 : updation]

Ex: a = Today is a good day

+ve indexing -ve indexing

Traversing from left to right:

+ve indexing: a =

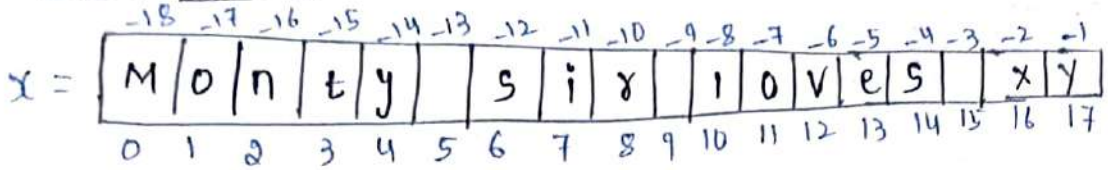
| -18 | -17 | -16 | -15 | -14 | -13 | -12 | -11 | -10 | -9 | -8 | -7 | -6 | -5 | -4 | -3 | -2 | -1 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| M | o | n | t | y | | s | i | r | 1 | 0 | v | e | s | | x | y | |
| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 |

a[we need loves

a[10 : 14+1 : 1]

10+1 = 11 == 15 (l)
11+1 = 12 == 15 (o)
12+1 = 13 == 15 (v)
13+1 = 14 == 15 (e)
14+1 = 15 == 15 (s)

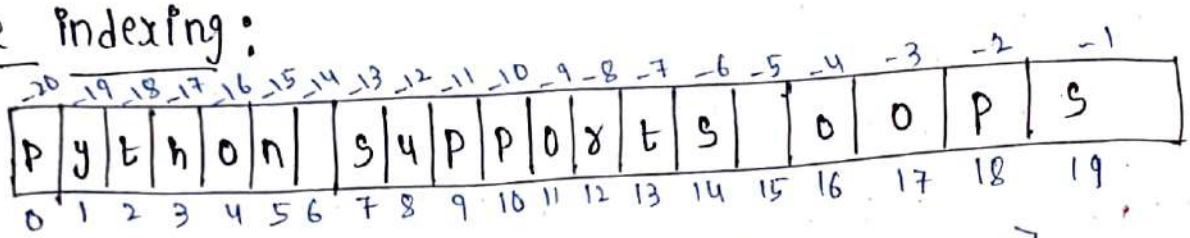
-ve indexing:




x[-8 : -H+1 : +1] ⇒ -8+1 = -7 == -3 (l)
-7+1 = -6 == -3 (o)
-6+1 = -5 == -3 (v)
-5+1 = -4 == -3 (e)
-4+1 = -3 == -3 (s)

Traversing from Right to left:

+ve indexing:




+ve: `Supports'

a[1H : 7-1 : -1]

1H-1 = 13 == +6 (s)
13-1 = 12 == 6 (t)
12-1 = 11 == 6 (r)
11-1 = 10 == 6 (o)
10-1 = 9 == 6 (p)
9-1 = 8 == 6 (p)
8-1 = 7 == 6 (u)
7-1 = 6 == 6 (s)

-ve: a[-6 : -13-1 : -1]

-6-1 = -7 == -1H (s)
-7-1 = -8 == -1H (t)
-8-1 = -9 == -1H (r)
-9-1 = -10 == -1H (o)
-10-1 = -11 == -1H (p)
-11-1 = -12 == -1H (p)
-12-1 = -13 == -14 (u)
-13-1 = -14 == -14 (s)

## List():

— A list is a collection of values, which can be:
- Homogenous (same type)
- Heterogenous (mixed types: int, float, str, bool, etc)

— values are enclosed with [ ]

64 bits Memory

Syntax:

```
var = [val1, val2, --- valn]
      (or)
var = list([val1, val2, --- valn])
```

Ex: a = [20, 0.1, 2+3j, [20, 10, 30], True]

Key features:

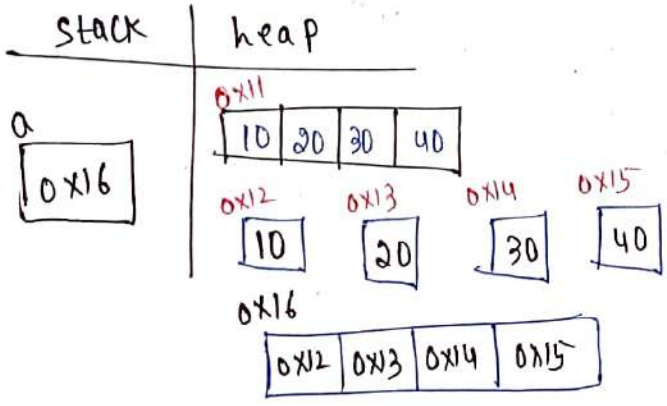
- ✓ Stores multiple values in a single variable
- ✓ Allows duplicate values
- ✓ supports different data types (heterogenous)
- ✓ supports nesting (list inside another list)
- ✓ Mutable — you can change, add, remove items

default value = ( ) (or) list ( )

64 bits Memory

- ✓ supports indexing
- ✓ Dynamical array

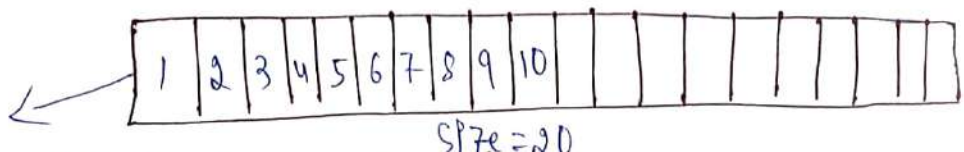
Ex: a = [10, 20, 30, 40]




Ex: 1 2 3 4 ⇒ 1 2 3 4 5 6 7 8 9 10

size = 10

if we want add "11" then size x 2 ⇒ 10 x 2 = 20




For this reason it is dynamic Array

Accessing : varname[index]

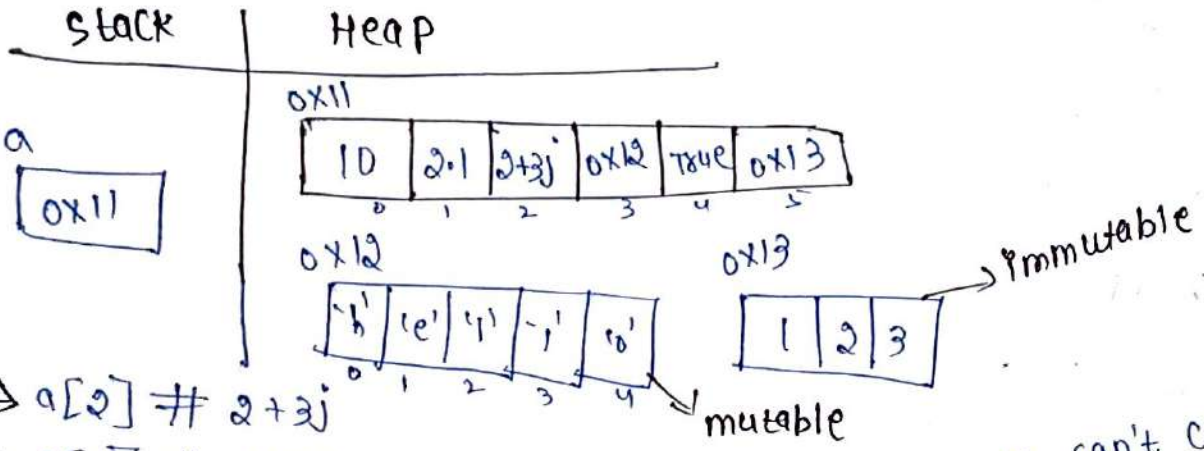
```
P = [10, 1.3, 2+3j, True, [10, 20, 30]]
     0    1     2     3      4
print(P[2])  # 2+3j
print(P[4][1]) # 20
```

Modifying : varname[index] = New value

```
P = [10, 20, 30]
P[1] = 200
print(P) # [10, 200, 30]
```

For Heterogenous :

```
a = [10, 2.1, 2+3j, 'hello', True, [1,2,3]]
```




→ a[2] # 2+3j

→ a[4] # True

→ a[3] # hello → for changing a[3][2] = 'z' → it can't change it instead of 'z' because it is mutable

```
a = [10, 20, [140, [60, 90], 110], 130]
     0   1   0     2   1     2     3
```

↓ change to 140
a[2][0] = 140

↓ 11
a[2][2] = 11

a[2][1][1] = 9

# LISTS INBUILT Methods:

1) append()
2) insert()
3) Extend()
4) POP()
5) remove()
6) split()
7) Join()

(1) append() → adds the value in a list at the end

syntax: varname.append()

Ex: a = [10, 'sai', 'bird', 2+5j, 2.5]
a.append(20)
a # [10, sai, bird, 2+5j, 2.5, 20]

(2) Insert() → adding values in a particular index in a list

syntax: varname.insert(index, value)

Ex: a = [10, 20, 30, 40]
a.insert(2, 70) # [10, 20, 70, 30, 40]
a.insert(700, 70) # [10, 20, 30, 40, 700] ↑ if there is no index then it store in last

(3) Extend() → Merging two list

syntax: Varname.extend(varname/list)

Ex: a = [1, 2, 3]
b = [4, 5, 6]
a.extend(b) # [1, 2, 3, 4, 5, 6]
(or)
a+b

(4) POP() → Removes value at the last
→ Remove value based on index

syntax: varname.POP(index) → default index value = -1

Ex: a = [10, 20, 30, 40] (0, 1, 2, 3)
a.POP() # 40 → It remove last value
a.POP(1) # 20 # [10, 30, 40]

a.pop(100) # through Error

(5) **Remove():** Remove first occurance value in a list

syntax: varname.remove(value)

ex: a = [10, 20, 30, 10, 40]

a.remove(10) # [20, 30, 10, 40]

↓ only 1 '10' will be remove, which is in 1st

(6) **split():** It return list

syntax: varname.split('char', no.of splits)

ex: a = 'Hi this is pranathi'

a.split() # Hi this is pranathi

a.split('i') # ['H', 'th', 's ', 's ', 'pranath']

a.split('i', 2) # ['H', 'ths', 'is', 'pranathi']

(7) **Join():** Joining

syntax: char for join.join(list)

ex: a = '@' b = ['hello', 'hi', 'bye']

a.join(b)

# hello@hi@bye

(or) '@'.join(b) # hello@hi@bye

(or) ' '.join(b) (or) a = ' '
a.join(b)

} If we need space blw words then

o/p: hello @ hi @ bye

# Tuple

— A tuple is a collection of values, which can be :
- Homogenous
- Heterogenous

— Values enclosed within ( )

Syntax :
```
var = (val1, val2, --- valn)
      (or)
var = val1, val2, --- valn
```

64 bits Memory

Ex: a = (20, 0.1, 2+3j, [20,10], True)

Key Features :

— Tuples allow duplicate values
— Tuples are immutable
— Tuples can contain nested structure
— slicing supported
— ordered datatype

Ex: a = 'hello' # packing.

a = 10, 20, 30
type (a) # <class tuples>

→ a = 10 it gives int
→ a = (10) it gives int
→ a = (10,) it gives tuple

Default value = ( ) or tuple( )

Ex: t = (1, 2, 3)
t[0] = 100 # error
because it can't change

| List | Tuple |
|---|---|
| 1) It is Mutable | 1) It is Immutable |
| 2) enclosed with [ ] | 2) enclosed with ( ) |
| 3) General purpose | 3) security purpose |
| 4) single value accepted | 4) single value accepted but by using comma |
| 5) It has more inbuilt method | 5) It has less inbuilt methods |

## Set :- set()

— A set is a collection of values, which can be:
  - homogenous
  - hetrogenous

— values are enclosed within curly braces { }

**Syntax :**

```
varname = {val1, val2, --- valn}
          (or)
varname = set({val1, --- valn})
```

64 bits memory

**Key Features :**

— set is mutable data type but set values must be immutable (ex: int, float, string, tuple)

— sets are unordered. — values are stored in random order

— No duplicates are allowed

— sets are unindexed

— Internally, sets are stored using hash table

— useful for data filtering, removing duplicates, membership tests.

Default value = set() considered false

— Nested not possible ex: {10, 20, {1, 2}} ✗

1) Default value :

```
s = set()
print(bool(s)) # False
```

2) Non-default value :

```
s = {0}
print(bool(s)) # True
```

3) Duplicate Elimination

```
s = {10, 20, 10, 30}
print(s) # {10, 20, 30}
```

4) Accessing Element :

Not allowed using indexing like s[0] → raise # error

Internal Working :

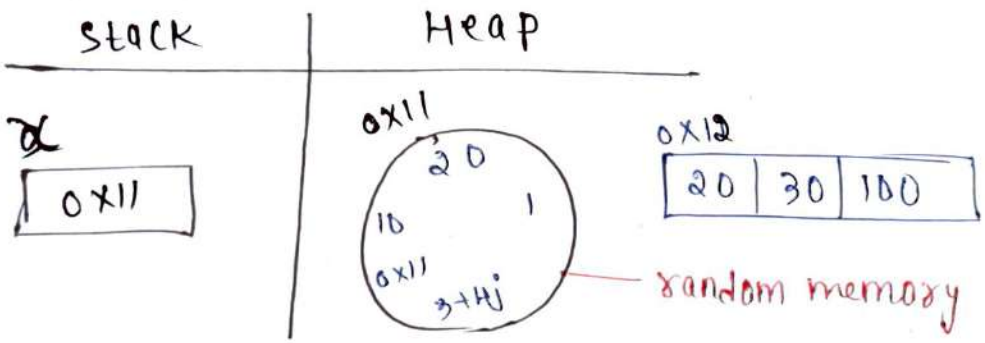
→ set stores values using a hash table

→ Each value is hashed, duplicates are ignored based on hash comparision.

Ex: x = {20, 10, 10, 1, (20, 30, 100), 3+4j, True}

print(x) # {1, 20, (20, 30, 100), 10, (3+4j)}

Note: Here True is equal to 1, so duplicates then only one is stored




## Set Inbuilt Methods :

1) add()
2) pop()
3) remove()

① add() — add element in a set in a random position

syntax : varname.add(value)

set — immutable

Ex: a = {10, 20, 2+4j, 'hello'}

a.add(30) # {10, 20, 2+4j, 30, 'hello'}

30 ↓ It will store in random position

② POP() — Remove random specified value

varname.POP

— ~~If the value is not present raises~~ the key error

Ex: a = {10, 20, 'hello', 3+4j}

a.POP() # {20, 'hello', 3+4j} → It randomly remove anyone number

a.POP() # {20, 3+4j}

Ex: {10, 0, True, 1, False}

a.POP(0) # {10, True, 1} → It will remove false also because False considered as '0'.

③ Remove() — (~~Removes~~ & returns an ~~arbitrary~~ (random) ~~element from the set~~)

— Remove values based on value.

— remove specified value

— If the value is not present raises the key error

varname.remove(value)

Ex: a = {10, 1.2, 4+6j, 'hello'}

a.remove(1.2) # {10, 4+6j, 'hello'}

# Dictionary (Dict):

* A Dictionary is a collection of key-value pairs.
* Enclosed within curly braces { }
* Think of it like a real dictionary : a word (key) and its meaning (Value)

**Syntax:**

```
varname = { key1 : value1, key2 : value2, ---- keyN = valuen }
```

- items
- colon
- mutable | immutable → SVDT, MVDT
- immutable → SVDT, SEY, tuple

```
varname = dict( key1 = value1, ---- keyn = valuen )
```

- follow identifier

**Ex:** dic = { 'a' : 'Apple' ; 'b' : 'ball', 'c' : 'cat' }

## Key Features

* Dict is mutable
→ keys should be always immutable
→ values should be immutable (or) mutable
→ dict is ordered datatype → After '3.7' it is ordered
→ Duplicates keys are not supported
→ values can be duplicate
→ key space is always visible layer in the memory

**Default value = 0**

→ An empty dict considered False

→ keys act as visible identifiers to access values directly.

Ex: d = {1:10, 2:20, 3:30, 4:40, 3:120, 5:40}

Only key will hashing in dict

| | Key | value |
|---|---|---|
| 0 | | |
| 1 | 4 | 40 |
| 2 | | |
| 3 | 5 | 40 |
| 4 | | |
| 5 | 2 | 20 |
| 6 | | |
| 7 | 1 | 10 |
| 8 | | |
| 9 | 3 | ~~30~~ 120 |

1 – hashing (1) ⇒ 2913682 % 10 ⇒ 7

2 – hashing (2) ⇒ 296789 % 10 ⇒ 5

3 – hashing (3) ⇒ 456789 % 10 → 9

4 – hashing (4) ⇒ 6789 % 10 → 1

Again

3 → hashing we get '9' but already the value is exist, so it will overide the value.

→ The value 40 again we have but there is no problem

Ex: d1 = {'a':10, 'b':20, 'c':30, 'a':40}

d2 = dict (a=[1,2,3], b={1,2,3}, c=(1,2,3), d='hello')

Stack space

d1 [0x10]

d2 [0x11]

Heap space

0x10

| key space | value space |
|---|---|
| a | ~~10~~ 40 |
| b | 20 |
| c | 30 |

| keyspace | valuespace |
|---|---|
| a | 0x12 |
| b | 0x13 |
| c | 0x14 |
| d | 0x15 |

0x12: | 1 | 2 | 3 |

0x13: | 1 | 2 | 3 |

0x14: (1, 2, 3)

0x15: | h | e | l | l | o |

d1['b'] # 20

d1['c'] # 30

d2['a'] # [1,2,3]

d2['a'][1] # 2

## Accessing value in dictionary using keys

Varname[key]

ex: d1['b'] # 20

Modify values in dict using / create New key & value in dict

```
Varname [key] = value
```

Ex: d1[b] = 200 # 200

d1 [e] = 100 # 100

- If key is present it will modify
- If key not present it will create new key & value

Ex: dic = {'a': 'Apple', 'b': 'Ball'}

dic ['a'] # Apple

dic ['a'] = 'Aeroplane # {'a': 'Aeroplane', 'b': 'Ball'}

dic ['c'] = 'cat' # {'a': 'Aeroplane', 'b': 'Ball', 'c': cat}

it will create because there is no 'c'.

## Dict Inbuilt Methods:

① get() ② POP() ③ setdefault()

① get(): To access data from dictionary

Ex: d = {'a': 10, 'b': 20, 'c': 30}

d ['c'] # 30

d ['d'] # error : for these reason we using get()

```
Syn: Varname.get(key, default value)
```

dict — None

Ex: d = {1: 10, 2: 20, 3: 30, 4: 40}

d.get (1) # 10

d.get (5) # none

→ If we don't need o/p as none, then we write as like

d.get (5, 'data not present') # data not present

② set default:

varname.setdefault(key, value)

Ex: d = {1:10, 2:20, 3:30}
d.setdefault(2, 40) # 20
d.setdefault(4, 40) # 40
↳ In the question there is no '4' key then it will add.
d # {1:10, 2:20, 3:30, 4:40}

③ POP(): Remove item based on key

varname.POP(key, default value)

Ex: a = {1:10, 2:20, 3:30, 4:40}
a.pop(3) # 30
a.pop(5) # key error
a.pop(5, 'value not found') # value not found

## Accessing elements from dict:

1) dict[key]
2) dict.get(key, [sp])
3) dict.keys() → [k1, k2]
4) dict.values() → [v1, v2]
5) dict.items() → [(k1, v1), (k2, v2)]

① dict[key]:

- If the key is present it will return the value
- If the key is not present it will raise the key error

Ex: d = {"a":10, "b":20}
d["a"] # 10
d["c"] # key error

2) dict.get(key, [sp]): → second parameter

- If the key is present will return the value
- If the key is not present it won't throw any error, but we can use a second parameter to print some stmts.

ex: d = {'a': 10, 'b': 30}

d.get('a') # 10

d.get('c') # not return anything

d.get('c', "invalid key") # invalid key

3) dict.keys():

— It will return list of keys

s = {'a': 30, 'c': 40}

s.keys() # dict_keys(['a', 'c'])

4) dict.values():

— It will return list of values

s.values() # dict_values([30, 40])

5) dict.items():

— It will return list of tuple of key, value

s.item() # dict_items([('a', 30), ('b', 40)])

## Concatination:

1) Concatination ('+') ← string + string
   list + list
   tuple + tuple

2) Concatination '|' (read as PI) — dict + dict

String + string

"jai" + "deva sir" # jai deva sir

storing

s = "jai"

s + "deva sir" # jai deva sir

(or)

s = s + "deva sir"

# 'jai deva sir'

2) list + list :

ex: l = [3,4]
l + [5] # [3,4,5]

ex: l = [3,4]
l = l + [5,6,7,8]
# [3,4,5,6,7,8]

Tuple + tuple :

ex: t = ("hi", "bye")
t = t + (5, "wifi") # ('hi', 'bye', 5, 'wifi')

ex: dict + dict → error
set + set → error

② concatination with '|'

dict + dict

ex: {1,2,3,4} | {5,6,7,4,8} # {1,2,3,4,5,6,7,8}

ex: d = {"a": 10}
d | {"b": 20, "a": 40} # {'a': 40, 'b': 20}

Removing Elements From Mutable datatype by clear & del

| | list | set | dict |
|---|---|---|---|
| clear() | l = [1,2,3]<br>l.clear()<br># none<br>l<br>[ ] | se = {1,2,3}<br>se.clear()<br># none<br>se<br>set() | d = {"a": 10}<br>d.clear()<br># none<br>d<br>{} |
| del | del var[index]<br>del var[start : stop±1 : up]<br>del var | se = {ele}<br>(or)<br>se - se | del dict[key] |

abs()

-ve numbers to +ve numbers

Ex: $n_1 = -111$

abs($n_1$) ## 111

$n_2 = 222$

abs($n_2$) ## 222

-abs()

+ve numbers to -ve numbers

ex: $n_1 = 323$

-abs($n_1$) ## -323

$n_2 = -323$

-abs($-n_2$) ## -323

## pass by value

1) Passing the value directly to the variable is called as pass by value

2) var = value

## pass by Reference

1) passing the reference address of one variable to the another variable is called as pass by reference

2) var = var

## Ascii (American standard code for information interchange)

— since machine cannot understand high level lang, the high level lang is convert into Ascii value by using Ascii value they will create their own binary members.

H.L.L ⟶ Ascii ⟶ binary

— Total Ascii values characters are 0-255

— Printable characters are 32-127

— Ascii digits 48-57

— Upper case 'A-Z' 65-90

— lower case 'a-z' 97-122

— Remaining are special characters

ord ("character") ----> ascii value

Ex: ord (" ") → 32
ord ("@") → 64
ord ("M") → 77

chr (ascii value) ----> character

Ex: chr (80) → 'X'
chr (97) → 'a'

## String Inbuilt Methods

(1) **upper()**: It converts lowercase to uppercase

Syntax: var.upper()

Ex: a = 'pranathi'
a.upper() # PRANATHI

(2) **lower()**: It converts uppercase to lowercase

Syntax: var.lower()

Ex: a = 'PRANATHI'
a.lower() # pranathi

(3) **swapcase()**: It converts upper to lower and lower to upper

Syntax: var.swapcase()

Ex: a='PRANATHI'
a.swapcase() # PRANATHI

(4) **capitalize()**: It converts first character of entire sentence to uppercase and remaining characters will be lowercase

Syntax: var.capitalize()

Ex: For capitalize
a = 'pranathI loves BIRDS'
a.capitalize()
# Pranathi loves birds

(5) **title()**: It converts first character of each word to upper case.

Syntax: var.title()

Ex: a = "pranathI shirisha sahithI anusha"
a.title()
# Pranathi Shirisha Sahithi Anusha

(6) **Index()**:

This method will return the index value of first matching element.

→ If the value is not present it will return value error

Syntax: var.index('value', [si], [ei]);

Ex: 'hyderabad biryani' = s
(0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16)
(-9 -8 -7 -6 -5 -4 -3 -2 -1)

s.index('a') # 5
s.index('a', 6) # 7
s.index('a', 8) # 14
s.index('a', 15) # value error

(7) rindex : this method will return the index value of last matching element.

— If the value is not present it will return value error.

Syntax : var.rindex('value', [st], [ei])

ex: s = hyderabad biryani
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16

s.rindex("i") # 16

s.rindex("i", 0, 16) # 11

s.rindex('i', 0, 11) # value error

(8) find & rfind :

find & rfind() methods are same as index & rindex method.

— Find method will give index value give first matching element

— rfind method will give index value give last matching element

— If the value are not there in index() and rindex() gives value error

— If the value is not present in find and rfind method gives "-1"

(9) Replace() : It is used to replace the old data with new data

Syntax : var.replace('old char', 'New', [count])

Ex:

s = Akshith is a good boy

→ s.replace(" ", '-')
# Akshith-is-a-good-boy

→ s.replace(" ", "_", 1)
# Akshith_is a good boy

→ s.replace(" ", "_", 2)
# Akshith_is_a good boy

→ s.replace("good", "lover")
# Akshith is a lover boy

→ s[0:10] # 'akshith is'

→ s[11:] # a good boy

• I need '-' in second space but not 1st space then we use concantination

s[0:10] + "_" + s[11:]
# Akshith is - a lover boy

## String boolean Function

These functions will return True | False

— we are having total "8" string boolean functions.

1) isupper()
2) islower()
3) isalpha()
4) isalnum()
5) isdigit()
6) isspace()
7) startswith(value)
8) endswith(value)

**isupper()**: It is the functions which checks all the alphabets are uppercase (or) not.

Syntax: var.isupper()

Ex: s = "PRANATHI"
s.isupper() # True
s = "PraNaTHI"
s.isupper() # False
s = "123"
s.isupper() # False
s = "@@@"
s.isupper() # False

**islower()**: It is the functions which checks all the alphabets are lowercase (or) not

Syntax: var.islower()

Ex:
"pranathi".islower() # True
"PRANATHi".islower() # False
"123@pr".islower() # True

**isalpha()**: It is the function which checks the char inside string are alphabet (or) not

Syn: var.isalpha()

Ex:
"ABC".isalpha() # True
"abA".isalpha() # True
"1aC2".isalpha() # False

**isalnum()**: It is the function which checks the char inside string are alphabet, num (or) not

Syntax: var.isalnum()

Ex:
'ABC'.isalnum() # True
'A2C4'.isalnum() # True
'@2A'.isalnum() # False

**isdigit()**: It is the function which checks the inside string are number (or) not.

Syntax: var.isdigit()

Ex: '123'.isdigit() # True
'A23'.isdigit() # False

**isspace()**: It is the function which checks the space blw the characters

Ex: "sai is good".isspace() # True
"sai".isspace() # False

**startswith()**: It is the function which will checks the string is starting with the specified value (or) not

Syntax: var.startswith(value)

Ex: s = "telangana state"
s.startswith("t") # True
s.startswith("T") # False
s.startswith("telangana") # True

endswith(): It is the function which will checks the string is ending with the specified value (or) not

Syntax: var.endswith()

Ex: s = "telangana state"
s.endswith("e") # True
s.endswith("t e") # True

## String Inbuilt Methods

| strip() | lstrip() | rstrip() |
|---|---|---|
| * used to remove the character from both the sides | * used to remove the charatter from left side | * used to remove the character from right side. |
| Syn: var.strip(value) | Syn: var.lstrip(value) | Syn: var.rstrip(value) |
| Ex: s = ' deva '<br>s.strip() # 'deva'<br>s1 = '@@deva@@'<br>s1.strip() # 'deva'<br>s2 = '@ deva sir@'<br>s2.strip() # 'deva sir' | Ex: s = ' deva '<br>s.lstrip() # 'deva '<br>s1 = '@deva@'<br>s1.lstrip() # 'deva@' | Ex: s = 'deva '<br>s1.rstrip() # 'deva'<br>s1 = '@deva@'<br>s1.rstrip() # '@deva' |

* strip(), lstrip(), rstrip() will remove spaces by default.

ljust(): It will help us to add the element towards right side

Syn: ljust(width, fill-char = " ")

Ex: s = "trainer name"
s.ljust(22) # 'trainer name '
s.ljust(22,'-') # 'trainer name----------'

rjust(): It will help us to add the element towards left side

Syn: rjust(width, fill-char = " ")

Ex: s = "trainer name"
s.rjust(22,'-') # '----------trainer name'

**center():** It will help us to add the element both the sides.

Syn: center(width, fill_char = " ")

Ex: s = "sub"

s.center(13) # '  sub  '

s.center(13, *) # '*****sub*****'

s.center(14, $) # '$$$$$$sub$$$$$$'

(or)

'$$$$$sub$$$$$$'

**split() & rsplit():** These two functions will help us to convert the sentence into list of words.

- Both of the functions will split at space by default.
- split will move from left to right
- rsplit will move from right to left.

Ex: s = "hi bye sky"

s.split() # ["hi", 'bye', 'sky']  s.split()[0] # 'hi'

s.split(" ", 1) # ["hi", 'bye sky']  s.split()[-1] # 'sky'

s.rsplit() # ['hi', 'bye', 'sky']

s.rsplit(" ", 1) # ['hi bye', 'sky']

**splitlines():** It help us to convert from paragraph to list of lines.

Syn: var.splitlines()

→ By Default it will split at '\n'

**count():** It will check how many times the specified element is repeated in the string.

Syn: var.count(value)

Write a prgm to check no.of characters, no.of words, no.of lines, no.of times the specified element is repeated?

```
s = ''' oh madhu oh madhu
naa manasu nadhi kaadhu
oh madhu oh madhu
naa manasu naaaaalo ledhu '''
```

→ len(s.split()) # 16

→ len(s.splitlines()) # 4

→ s.count(" ") # 12

→ s.count("oh") # 4

→ len(s) # 84

**Join():** It is used to join the elements of the collection.

— the element inside the collection should be in the form string

Syn: "sir".join(iterable)

ex: l = ["rahul sir", "anushka shetty"]

"loves".join(l) # 'rahul sir loves anushka shetty

→ l = [1, 2, 3]

"@".join(l) # Error because inside list there is a number.

## Type casting

converting from one data type to another data type is called as type casting

— To achieve type casting we use class objects.

## Class obj's

- int()(i)
- float()(f)
- complex()(c)
- bool()(b)
- str()(st)
- list()(l)
- Tuple()(t)
- set()(se)
- delt()(d)

## NOTE

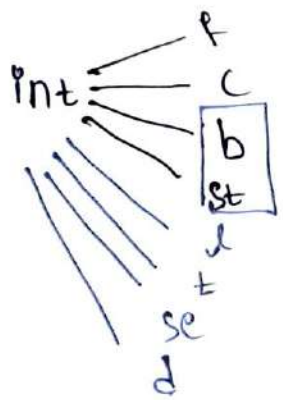
- we can convert any data type to boolean & string

→ boolean

| True | False |
|---|---|
| if we have any value | if we have default value (or) empty data |

→ string – we can have any character in string

→ list | Tuple | set | dict – we cannot convert primitive data type to the list | Tuple | set | dict because they convert only iterable (NIPDT)




ex: #int
n=18

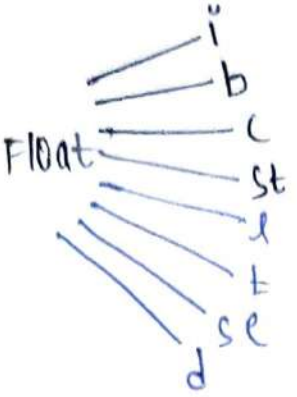
→ float(n)
# 18.0

→ complex(n)
# (18+0j)

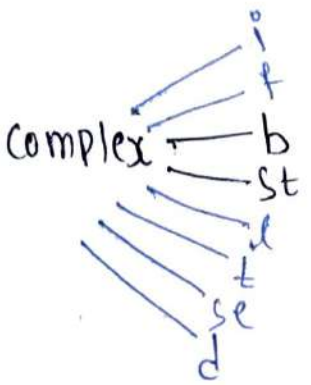
→ bool(n)
# True

→ bool(int()) → default value
# False

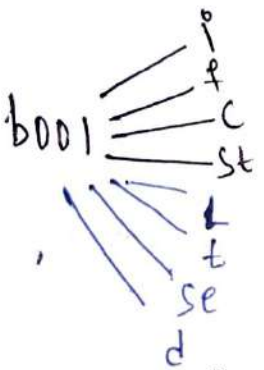
→ str(n)
# '18'




ex: # float
f = 1.2
→ int(f)
# 1
→ complex(f)
# (1.2+0j)
→ bool(f)
# True
→ bool(0.0)
# False
→ str(f)
# '1.2'




ex: # complex
c = 3+5j
→ bool(c)
# True
→ str(0j)
# False

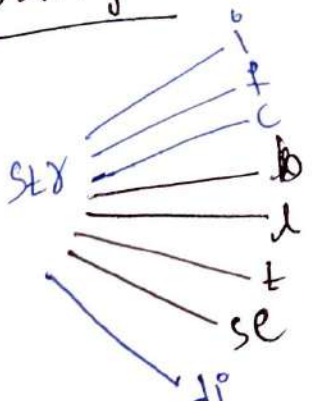



ex: # bool
t = True
→ int(t)
# 1
→ float(t)
# 1.0
→ complex(t)
# 1+0j
→ str(t)
# 'True'

→ Black lines are possible

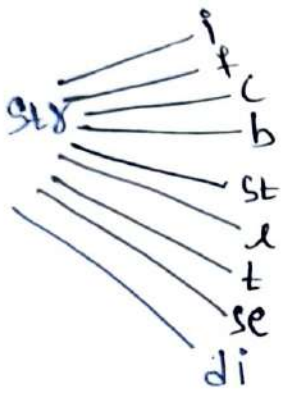
→ Blue lines are not possible

## String with ALPHA




ex: s = "aba"
→ bool(s)
# True
→ bool("")
# False
→ list(s)
# ['a', 'b', 'a']
→ tuple(s)
# ('a', 'b', 'a')
→ set(s)
# {'a', 'b'}

## String with only number (int)




Ex: s = "28"

→ int(s)
# 28

→ float(s)
# 28.0

→ complex(s)
#(28+0j)

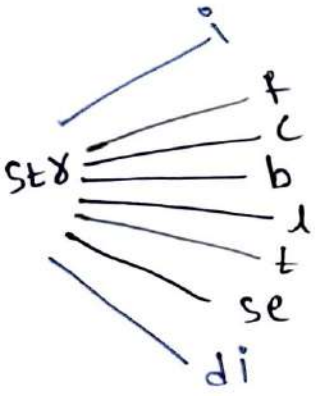
→ bool(s)
# True

→ list(s)
# ['2', '8']

→ tuple(s)
# ('2', '8')

→ set(s)
# {'8', '2'}

## String with number & special char (float)




Ex: s = "1.3"

→ float(s)
# 1.3

→ complex(s)
#(1.3+0j)

→ bool(s)
#True'

→ list(s)
# ['1', '.', '3']

→ tuple(s)
# ('1', '.', '3')

→ set(s)
# {".", "1", "3"}

DT. 4/4/2(

print()

To get the output we use print statement

syn

print ("value "/var, sep = " ", end = "\n")

Ex:-

```
name = "deva sir"
age = 22
print (age)
```

o/p

devasir

22

Ex:-

```
name = "davasir"
age = 22
gf = none
print(name, end=", ")
print (age, end = "_ ")
print(gf)
```

o/p

devasir, 22 - none

Ex:-

```
name = "rahul sir"
wife = "anushka"
children= 1
Marrag_date = "30-4-2027"
Print(name, wife, sep = ", " end = "_ ")
print (children, end = " | ")
print (marrag_date)
```

rahul sir, anushka - 1 | 30-4-2027

Ex:-

```
name = "rahul sir"
sub = "web"
print (name, sub)
print (name, sub, sep=", ")
```

op:-

rahul sir web

rahul sir, web

6/4/26

# Input

→ To Take the user input/data we use input function
→ By default it will take string datatype.
→ syn:-

```
var = input("placeholder:")
```

Examples

```
n = input("enter the n value:")
n1 = input("enter the n1 value:")
print(n+n1)
# 3345
```

ex:

```
n = int(input("Enter the n value:"))
n1 = int(input("Enter the n1 value:"))
print(n+n1)
# 111
```

Ex:-

```
n = float(input("Enter the n value:"))
n1 = float(input("Enter the n1 value:"))
print(n+n1)
# 3.0
```

## eval

→ When we don't know the what value the user is going to pass we use "eval"
→ " " are mandatory
ex:

```
n = eval(input("enter the n value:"))
n1 = eval(input("enter the n1 value:"))
print(n+n1)
```

formatted string

To add the statement along with the values we use F-string.

syntax

F/f"statement {var/value} statement {var./value}"

```
trainer = "pavan sir"
print (f" @SP trainer :{trainer}")


amount = eval (input("enter the amount :"))
print (f"Rs.20.{amount} rupailu phonepay
                    dwara vachayi")
```

Control statements

. To control the flow of execution we use control statements

. The control statements are classified into three types they are

* conditional statement
* looping statement
* loop control statement.

conditional statement

To check and apply condition we use conditional statment. they are classified into

. if
. else
. elif
. nested if
. match case

looping statement

To repeat a task for multiple times we use looping statement. There are classified into into.

- while
- for

loop control statement

To control the loop we use loop control statement They are classified into

- pass
- break
- continue

## Decisional / conditional statement

To apply the conditions we use conditional statement. The set of statements will execute based on the decision for only one time.

1. Simple if statement

If keyword will check the final result is True or false. If it True the true block will be executed.
otherwise it will send the controller to either else or elif or not of the entire block.

if condition / expression
.set of statement
where if is predefined keyword
condition / expression is user defined.

## else statement

- whenever we have one condition and two decisions are there then 'if else' is used.
- if condition is satisfied then true statement block (TBS) is executed, otherwise controller is sent out of the if block and false statement block (FBS) is executed (else)

## If else statement

```
if condition expression:
    . set of statements (TBS)

else:
    . False Block statement (FBS)
```

→ where if is pre-defined keyword
→ condition/expression is user defined
→ else is pre-defined keyword.

## Examples

WAP to create check the number is even or not. If even, get the square else get the cube.

```
n = int(input("enter the number:"))
if n%2==0:
    print(n**2)
else:
    print(n**3)
```

WAP To check char is uppercase or not. # if yes convert it to lowercase else ignore

```
var="8"
if "A" <= var <= "Z":
    print(chr(ord(var)+32))
```

```
else:
    print("it is not a uppercase")
```

WAP to character is vowel or not
# if yes Add it to the list else add ito string.

```
var = input("enter the value:")
vowel = []
no = ""
if var in "AEIOUaeiou":
    vowel.append(var)
    print(vowel)
else:
    no += var
    print(no)
```

WAP to check palindrome or not

```
s = input("enter the data:")
if s == s[::-1]
    print("palindrome")
else:
    print("not a palindrome")
```

WAP to check it is a mutable data or not

```
data = eval(input("enter the data:"))
if type(data) in (list, set, dictionary):
    print("mutable")
else:
    print("immutable")
```

## if else if / elif

- In this statement we can write one or more than one condition
- one if statement should be present.
- elif, we can write 'n' no.of elif statements based on situation
- else statement is optional.

```
if condition:
    ↓
  [T.B.S]

elif condition:
    ↓
  [T.B.S]

else:
    ↓
  [F.B.S]
```

WAP to check number is +ve or -ve or neutral

```
n = int(input("enter the number:"))
if n>0:
    print("positive")
elif n<0:
    print("negative")
else:
    print("neutral")
```

check the char is uppercase or lower or digit or special character

A] With builtin function

```
data = input("enter the data:")
if data.isupper():
    print("uppercase")
elif data.islower():
    print("lowercase")
elif data.isdigit():
    print("digit")
else
    print("special character")
```

## Nested if

To create a condition inside another condition we will use nested if

if condition : F → elif condition : F → else :

```
if condition
    ↓
    TBS
elif condition
    ↓
    TBS
else:
    FBS
```

```
if condition
    ↓
    TBS
elif condition
    ↓
    TBS
else:
    FBS
```

```
if condition
    ↓
    TBS
elif condition
    TBS
else:
    FBS
```

wap to get second greatest number among 3 numbers.

```
n1 = int (input ("enter n1 value:"))
n2 = int (input ("enter n2 value:"))
n3 = int (input ("enter n3 value:"))
if n1>n2 and n1>n3
    print (f"n2 : {n2} is the second greatest")
elif n3>n2:
    print (f "n3 : {n3} is the second greatest")
else:
    print ("enter diff number")
elif n2>n1 and n2>n3:
    if n1>n3:
        print (f"n1 : {n1} is the second greatest")
    elif n3>n1:
        print (f "n3 : {n3} is the second greatest")
    else:
        print ("enter diff number")
elif n3>n1 and n3>n2:
    if n1>n2:
        print (f"n : {n1} is the second greatest")
    elif n2>n1:
        print (f "n2 : {n2} is the second greatest")
    else: print ("enter diff number")
else:
    print ("Boss enter diff number")
```

# Looping statements

Looping statements are used to execute a block of code repeatedly.

- for loop
- while loop

## For loop purpose

Iterates over a sequence (eg. a list, tuple or string) or other iterable objects.

## while loop purpose

Repeatt a block of code as long as specified condition is true.

### syntax

- initialisation
- condition
- update (increment / decrement)

When we don't know how many time the iteration to be done at that time we will go for the while loops.

Here we have to track the number iteration

Here we have to increment / decrement reference variable manually

Ex:

```
s=1
e=3
while s<=3
print(s,end=" ")
st=1
```

| s | e | s<=e | o/p | inc(s) |
|---|---|---|---|---|
| 1 | 3 | 1<=3 (T) | 1 | 1+1=2 |
| 2 | 3 | 2<=3 (T) | 2 | 2+1=3 |
| 3 | 3 | 3<=3 (T) | 3 | 3+1=4 |
| 4 | 3 | 4<3 (F) | | |

```
s=3
e=0
while s>=e:
print(s,end=" ")
s=s-1
```

| s | e | s>=e | o/p | dec |
|---|---|---|---|---|
| 3 | 0 | 3>=0 (T) | 3 2 1 0 | 3-1=2 |
| 2 | 0 | 2>=0 (T) | | 2-1=1 |
| 1 | 0 | 1>=0 (T) | | 1-1=0 |
| 0 | 0 | 0>=0 (T) | | 0-1=-1 |
| -1 | 0 | | | |

WAP to print 0-5 even numbers.

```
s=0
e=5
while s<=e:
    print(s)
    st=2
```

WAP to print 1-5 odd numbers.

```
s=1
e=5
while s<=e:
    print(s)
    st=2
```

WAP to print 'A'.....'Z'

```
s=65
while s<=90:
    print(chr(s),end=" ")
    st=1
```

WAP to print 'z'.....'a'

```
s=122
while s>=97:
    print(chr(s),end=" ")
```

WAP to print vowels from 'A' ---'Z'

```
s=65
while s<=90:
    if chr(s) in "AEIOU":
        print (chr(s))

    s+=1
```

WAP to get A-Z consonant.

```
S=65
while S<=90:
    if chr(s) not in "AEIOU":
        print (chr(s))

    s+=1
```

WAP to get 1-10 even numbers

```
s=1
e=10
while s<=e:
    if(s%2==0)
    print(s, end=" ")

    s+=1
```

WAP to get 0-10 odd numbers.

```
s=0
while s<=10
    if (s%2==1):
        print(s, end=" ")

    s+=1
```

Q] input 127 o/p: 10

```
n=127
total=0
while n>0:
    last=n%10
    total+=last
    n//=10
print(total)
```

8] input : 127 o/p: 14

```
n=127
pro=1
while n>0:
    last=n%.10
    pro *= last
    n//=10
print(pro)
```

6] WAP to get alternate character

```
data="sky"
s=0
e=len(data)-1
while s<=e:
    print(data[s])
```

count no of upperclase

```
data="HELLO"

soln s=0
    e=len(data)-1
    count_=0
    while s<=e:
        if "A"<=data[s]<="Z":
            count_+=1
        s+=1
    print(count_)
```

9] WAP to get each character

```
data="sky"
s=0
e=len(data)-1
while s<=e:
    print(data[s])
    s+=1
```

WAP to get vowels from a given string

```
data="HELLO HeLLO"
o/p :"EOeO
s=0
e=len(data)-1
res=""
while s<=e:
    if data[s] in 'AEIOUaeiou':
        res+=data[s]
    s+=1
print(res, type(res))
```

Count no of uppercase | lowercase | digit | special cha

data = "HeLLo@56 Hi *7 73"  # 3, 4, 5, 4

```
s = 0
up = 0
lc = 0
di = 0
sc = 0
e = len(data) - 1
while s<=e
    if "A" <= data[s] <= "Z"
        up += 1
    elif "a" <= data[s] <= "z"
        lc += 1
    elif "0" <= data[s] <= "9"
        di += 1
    else
        sc += 1
        s = s+1
print (uppercase)
print (lowercase)
print (digit)
print (special character)
```

Logic of swapcase()

Reverse the string

```
data = "MOM"
s = len[data] - 1
e = 0
res = ""
while s >= 0:
    res += data[s]
    s -= 1
print (res)
```

o/p: MOM

15/4

palindrome or not

```
data = input ("enter the data:")
s = len (data) - 1
e = 0
res = ""
while s >= 0
    res += data[s]
    s -= 1
if data == res
    print ("palindrome")
else:
    print ("not palindrome")
```

logic of upper function

```
data = "Hi@I"
s = len(data)-1
e = 0
res = ""
while s<=0:
    if "a"<=data[s]<="z"
        res += chr(ord(data[s])-32)
    else:
        res += data[s]
    s+=1
print(res)
```

logic of lower function

```
data = "Hi@I"
s = len(data)-1
e = 0
res = ""
while s<=0:
    if "A"<=data[s]<="Z
        res += chr(ord(data[s])+
    else:
        res += data[s]
    s+=1
print(res)
```

## ARRAY

Sum of an array

```
data = [3, 5, 1]
s = 0
e = len(data)-1
total = 0
while s<=e:
    total += data[s]
    s+=1
print(total)
```

MAX number from an array

```
data = [5, 6, 7, -1, -2]
if data == []
    print("empty")
else:
    assume = float(-inf)
    s = 0
    e = len(data)-1
    while s<=e:
        if data[s]>assume:
            assume = data[s]
        s+=1
    print(assume)
```

min number from an array

```
data = [5, 6, 7, -1, -2]
if data == []
    print("empty")
else:
    assume = float(inf) #data[0]
    s = 0
    e = len(data)-1
    while s<=e:
        if data[s]<assume:
            assume = data[s]
        s+=1
```

Average of an array

```
data = [3, 5, 1]
s = 0
e = len(data)-1
n = len(data)
sum = 0
Avg = 0
while s<=e:
    sum += data[s]
    s+=1
avg = sum/n
print(avg)
```

reverse of an array

```
data = [3, 5, 6, 7]
s = len[data] - 1
c = 0
ra = []
while s >= 0
    res += [data[s]]
    s -= 1
print(res)
```

Infinite loop in while

```
name = input("enter the data:")
while True:
    print(f"my name is {name}")
```

## FOR LOOP

→ It is used when the number of iteration to be done is known.

→ No need to increment the reference variable manually

→ There is no need to keep the track of iteration for loop do it implicitly.

syntax

```
for ref_var in collection:
```

(kw: for; Identifier: ref_var; membership operator: in; NLP datatype (value): collection)

(or)

```
for ref_var in built in methods:
```

(kw: for; Identifier: ref_var; membership operator: in; NLP datatype: built in methods)

## For loop

* A forloop is used when you want to iterate over a sequence

* for loop is used when the number of iterations is known in advance.

* for loop requires a sequence to iterate over.

* only condition we need (some time initialization)

* forloop is typically used for iterating over a fixed sequence of items

* for ref_var in iterable

for ref_var in builtin methods

## while loop

* when we want to repeat a block of code until certain condition is met.

* while loop is used when the no.of iterations is not known in advance

* while loop requires an initial condition that is tested at the begining of the loop.

* we need initialization, condition and incrementation

* while loop is used for more complex control flow situations

* 1. Initialization
  2. while condition
  3. Increment/decrement.

WAP to iterate over a string and get ele one by one

```
s = "hello"
for char in s:
    print(char)
```

o/p:-
h
e
l
l
o

WAP to iterate over a list and getele one by one

```
l = [2, "indu", 8.4]
for ele in l:
    print(ele)
```

o/p:-
2
indu
8.4

WAP to iterate over a tuple and get ele one by one

```
t = (2, "suma", 10.0)
for ele in t
```

o/p:-
2
suma
10.0

WAP to iterate over a set and get a ele one by one

```
se = {"hello", "car", "murc"}
for ele in se:
    print(ele)
```

WAP to iterate over a dictionary and get one by one

=> d = {"a":1, "b":2}  o/p:- a
                           b

```
for ele in d:
    print(ele)
```

=> d = {"a":1, "b":2}  o/p:- 1
                            2

```
for ele in d:
    print(d[ele])
```

=> d = {"a":1, "b":2}  o/p:- a 1
                            b 2

```
for ele in d:
    print(ele, d[ele])
```

DT: 14/4/26

For loop with builtin functions

# values

get all the values from the given dict:

```
d = {"a":1, "c":6}
for v in d.values(): # for v in [1,6]:
    print(v)
```

get all the pair from the given dict

```
d = {"a":1, "c":6}
```

#items

unpacking ele in for loop block.

```
for p in d.items():
    print(p[0], p[1])
```

o/p: a 1
c 6

unpacking ele within for loop declaration

```
d = {"a":1, "c":6}
for k, v in d.items():
    print(k,v)
```

#split

WAP to get the words from a given string one by one

```
s = "hyderabad biryani vere level"
for w in s.split
    print(w)
```

WAP to get the palindrome words from a given string

#palindrome word -> level

```
for w in s.split():
    if w[::-1] == w:
        print(w)
```

WAP to convert the lower case to uppercase and add them to the last list.

```
l = []
for w in s.split():
    l.append(w.upper())
print(l)
print(" ".join(l))
```

# Range function

↳ range() ----> inbuilt class ----> iteration object
↳ Range always generates numbers
↳ Returns a range object, sequence of numbers, starting from 0 (by default)
↳ ---> and increment by 1 (by default) and ends at the specified number.
↳ to loop through a set of code a specified no. of times, we use range function
↳ start index is always included and, end index is excluded
↳ start=0, end = mandatory otherwise it will throw error
↳ Syntax: range (start, stop, [step])

```
print numbers from 0 to 6 using range()
for n in range (0, 7, 1)
    print (n, end = " ")
print numbers from 1 to 10 using range()
for n in range (1,11,1)
    print(n, end = " ")
print numbers from 0 to 20 with a step of 2.
for n in range (0, 21, 2)
    print (n, end = " ")
Print numbers from 10 to 1 in reverse order
for n in range (10, 0, -1)

print number all even numbers from 2 to 20
for n in range (2, 21, 2)
```

WAP to get alternate ele from the string by using for loop.

```
data = "sky"
for i in range(0, len(data), 2):
    print(data[i])
```

o/p:-
s
y

WAP to get the output as "HLO" with input: "Hello"

```
data = "hello"
res = ""
for i in range(0, len(data), 2)
    res += data[i].upper()
print(res)
```

o/p:-
=
HLO

WAP of reverse a string

```
data = "python"
res = ""
for i in range(len(data)-1, -1, -1):
    res += data[i]
print(res)
```

5-1 = len(d) - 1 = 5 - 1 = 4
0 - 1 = -1
-1 → reverse dire

print index number and char of the string.

```
data = "hello"
res = []
for i in range(0, len(data), 1):
    res.append((i, data[i]))
print(res)          # o/p:- [(0,'h'), (1,'e'), (2,'l'), (3,'l'), (4,'o')]
print(dict(res))    # o/p: {0:'h', 1:'e', 2:'l', 3:'l', 4:'o'}
```

Count no:of characters

```
data = "hello"
#data = "hi"
char_count = 0
for i in data:
    char_count += 1
print (char_count)
```

count no of upper

```
s = "hELLO BYE"
uc_count = 0
for i in s:
    if "A"<= i <="Z":
        uc_count += 1
print (uc_count)
```

WAP for s = "@hEL3Lo 5BYE99"

#uc = 5 , lc = 3, dig = 4 spcl = 2

```
s = "@hEL3Lo 5BYE99"
uc = 0
lc = 0
dig = 0
sc = 0
for i in s:
    if "A" <= i <= "Z":
        uc_count += 1
    elif "a" <= i <= "z":
        lc_count += 1
    elif "0" <= i <= "9":
        dig_count += 1
    else
        sc_count += 1
print (uc_count)
print (lc_count)
print (dig count)
print (sc_count)
```

WAP for s = "HEl@3 o/p:- hELE@3

```
data = "HEl@3"
```

WAP for l = [1,2,3] o/p: [3,2,1]

```
l = [1,2,3]
res = []
for i in range (len(data),-1,-1)
    res += data[i]
print (res)
```

WAP for t = (3,4,6) o/p: (6,4,3)

```
t = (3,4,6)
res = ()
for i in range (len(data),-1,-1)
    res += t[i]
print (res)
```

WAP l = [1,2,3,4,8]

```
l = [1,2,3,4,8]
res = []
for i in range (0, len(data), 2)
    res += l[i]
```

WAP l = [1,2,3] #3 , l = [1,2,3,4,8] #5

## pattern

```
out=1
while out<=4
    i=1
    while i<=out
        print(i, end=" ")
        i+=1
    print()
    out+=1
```

o/p:-

1
1 2
1 2 3
1 2 3 4

```
col=1
while col<=4
    i=1
    while i<=5
        print("*" end="")
    print()
    col+=1
```

o/p:-

* * * * *
* * * * *
* * * * *
* * * * *

## Frequency

Tracking repetation of the element

s="aabcab"

| char | repeat |
|---|---|
| a | 3 |
| b | 2 |
| c | 1 |
| key | value |

Note:- d={}

dict[key]=d['a']=1

d
{'a':1}

dict[key]->d['a']+=1

Ex:-

s="abac"

d={} # {a:1, b:1, c:1}

```
for ch in s:
    if ch not in d:
        d[ch]=1
    else:
        d[ch]+=1
print(d)
```

the input is a="abac" o/p: "a2 b1 c1"

a="abac"

d={}

~~for d in d:~~

```
for ch in a:
    if ch in d:
        d[ch]+=1
    else:
        d[ch]=1
for k,v in d.items():
    res=k+str(v)
    print(res)
```

The input is duplicate char a="abac" o/p: a

```
a="abac"
d={}
for ch in a:
    if ch not in d:
        d[ch]=1
    else:
        d[ch]+=1
for k,v in d.items():
    if v>1:
        print(k)
```

o/p

a
c

## Loop control statement

pass:- used to passkip the statement

break:- used to stop the loop

continue:- used to skip the current iteration

=> while

```
s=1
e=5
while s<=e:
  pass
```

Break

=> while

```
s=1
e=5
while s<=e:
    if s==4      # s==4 (T)
      break      # ----> exit
    print(s,end=" ")
    s+=1
```

=> for

```
for i in range(1,11)
  ...
```

=> for

```
for i in range(1,11)
  if i==6:
      break
  print(i,end=" ")
```

# Exception handling

Exception:- The interruption caused during the flow of execution of the program. also known as error.

Exceptional handling:-

It is the way to handle the exceptions or errors.

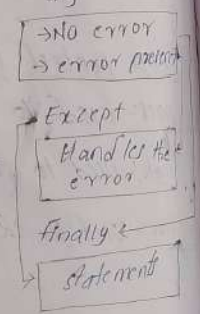
How to handle error:-

To handle errors we use 2 blocks

Try:- checks if error is present or not

except:- Handles the error if error present

finally:- Execute statement.




Types of except block

Default:- Handles all types of errors

syn:-

```
try:
  checks error
except:
  handles error
```

specific:- It handles the error if specified error match is the error in try block.

Multiple:- When you are confused between multiple errors we can use multiple except block

Generic:- To overcom the drawback of multiple except block we use generic except block

Types of Exceptions

| compile time | Run time |
|---|---|
| ↳ occurs during compilation time | ↳ occurs during the run time |
| ↳ mostly syntax errors | ↳ logical errors like [Name, type, value, key, Index] |
| ↳ used by or done by users and programmers. | ↳ Done by users or programmers. occurs in logics called as logical errors |
| ex: syntax, indentation errors. | |

Try block:-

↳ code that might raise an exception is placed in the try block.

↳ If error occurs, we move to the except block.

↳ else line is executed in try block

Except block

↳ If exception occurs in try block, python searches for matching except block.

-> The code inside the except block is executed if the corresponding matches error found in try block.
-> Except block will execute only when exception is occurs
-> Here we will handle the exception.

finally block
-> If the error is present or not, finally block will always get executed.
-> It is often used for printing same statement.

Note:
We cannot handle syntax error and indentation error

Alias Name
-> as keyword
-> Syntax:
except ErrorName as Error_msg:
[Error msg]

Ex: Raise keyword
age = int(input("enter your age:"))
if age >=18:
print(f"{age} eligible")
else:
raise exception("not eligible")

Alias Name
-> as keyword
-> Syntax:
except ErrorName as Error_msg:
[Error msg]

as keyword
try:
name = input("enter your name:")
age = int(input("enter your age:"))
mobile = int(input("enter your mobile:"))
except ValueError as e_msg:
print(f"Error handled:{e_msg}")

Creating the error

```
raise ErrorName ("error Msg")
```

ErrorName → any → if we don't know we can use exception (Builtin class)

Raise keyword

```
age = int (input ("enter your age:"))
if age > 18:
    print ("eligible")
else:
    raise exception ("not eligible")
```

## Max

Syntax:-

```
max("homogeneous iterable")   #
print (max ("hello"))          # o
print (max ([1, 6, 3, 1]))     # 6
print (max ((1, 13, 4, 17)))   # 17
print (max ({77, 36, 48, 96}))  # 96
print (max ({"a": 25, "b": 10}))  # b  # keys
print (max ({"a": 25, "b": 30}.values()))  # b
```

## Min

Syntax:-

```
min ("homogenew iterable")
```

Ex:-

```
print (min ("hello"))          # e
print (min ([1, 6, 3, 1]))     # 1
print (min ((1, 13, 4, 17)))   # 1
print (min ({77, 36, 48, 96}))  # 36
print (min ({"a": 25, "b": 30}))  # a
print (min ({"a": 25, "b": 30}.values()))  # 25
```

## Sum

Syn

```
sum("heterogeneous iterable")
print (sum ([1, 6, 3, 9]
print (sum ((4, 5, 6, 8, 9)))
print (sum ({55, 22, 66, 44}))
print (sum ({"a": 3, "z": 6}))  # Type error - int+str
print (sum ({"a": 3, "z": 6}.values()))
```

write a program input : s = "1r2p35" o/p : 11

```
s = "1r2p35"
sum_ = 0
for ch in s:
    if "0" <= ch <= "9":
        sum_ += int(ch)
print (sum_)
```

## Sorted

→ It is used to arrange elements in order

→ It doesnot change the original data - it creates a new sorted result

→ Syntax

sorted (iterable, key=not ; reverse = True | False)

True → Asce, False → desce

parts

→ Iterable : the data you want to sort

→ key : (optional) a rule to decide how sorting happens

→ reverse True for Ascending and False for Descending order

Ex:-

```
l = [1, 6, 3, 8]
print (sorted(l))
print (sorted(l), reverse = True))

t = (8, 2, 8, 3)
print (tuple(sorted(t)))
print (tuple(sorted(t, reverse=true)))

se = {7, 2, 8, 9}
print (sorted(se))
print (sorted(se), reverse = True))

d = {"b": 7, "a": 120, "z": 180}
print (sorted(d))
print (sorted(d, reverse = True))

# Values:-
print (sorted (d.value()))
print (sorted (d.values(), reverse= True))

# second max
l = [1, 3, 5, 2, 8, 4, 9]
print (sorted(l)[-2])

# Third min
l = [1, 3, 5, 2, 8, 4, 9]
print (sorted(l)[1])

s = "hello"
print (sorted(s))
# print (sorte d (d), reverse = False)
"".join (['e', 'h', 'l', 'l', 'o'])

s1 = "bye"
print (sorted(s1), re.
```

# FUNCTIONS

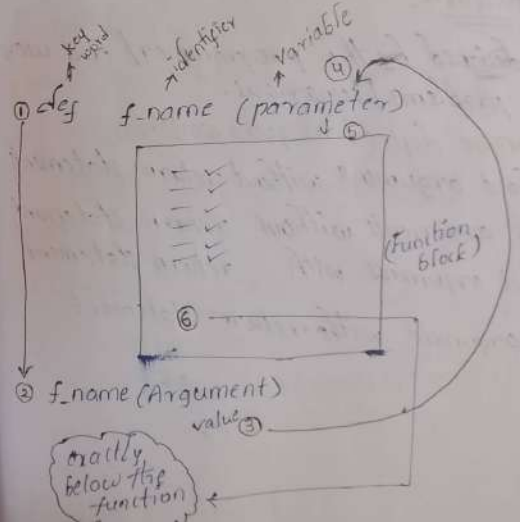
**Function:-** A function is a block of code which will execute whenever it is called

**Note:-**

* As a program grows larger, function make it more organized and mangeable
* Function avoids repetition and makes the code reusable
* functions allow you to breakdown a complex program into smaller, mangable:
* Changes or updates can be made more easily when code is organised into functions

**Advantages**

- Reusability
- Readability
- Easy to modify
- Easy to maintain
- Easy to debug



## classification of function

The functions are classified into two types:
→ predefined
→ userdefined

### predefined:

→ These are designed by the python developers.
→ Each pre-defined function will perform a specific unique task

Types:-
(i) pre-defined utility functions
(ii) String built-in functions
(iii) List built-in functions
(iv) Tuple " "
(v) set " "
(vi) Dictionary " "

### Userdefined function:

These are designed by the programmer/ end user for real time problems/scenarios.
Userdefined can be defined in four ways:-
(i) Functions without arguments without return statement
(ii) Functions with arguments without return statement
(iii) Functions without arguments with return statement
(iv) Functions with argument with return statement

```
def greet(name):
    print(f"Good afternoon:{name}")
greet("indu")
greet("suma")
greet("anagha")
```

o/p:-
Good afternoon indu
Good afternoon suma
Good afternoon anagha.

WAP for even or not using function

```
def is_even(n):
    if n%2==0:
        print(True)
    else:
        print(False)
```

or

```
print(True if n%2==0 else False)
```

```
is_even(5)
is_even(6)
```

WAP for odd or not

```
def is_odd(n):
    print(True if num%2==0 else False)
is_odd(7)
is_odd(8)
```

square and cube

```
def square_cube(n):
    print(f"sq:{n}, cube:{n}")
square_cube(2)
```

palindrome or not

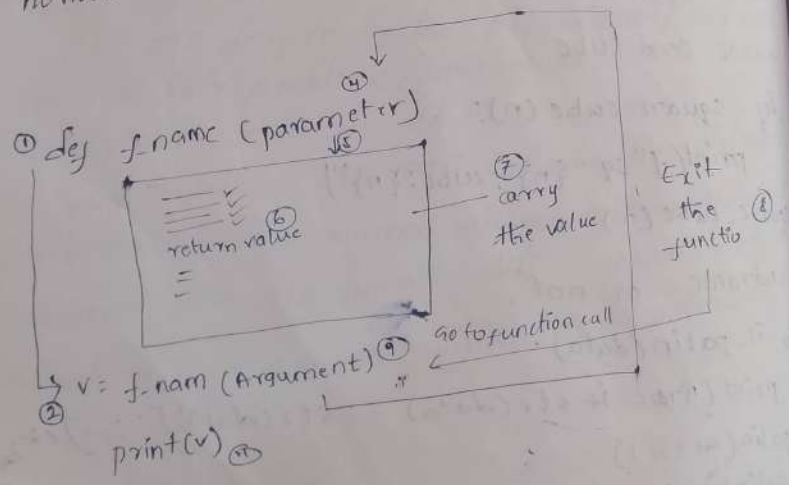
```
def is_palin(data):
    print(True if str(data)==str(data)[::-1] else False)
is_palin(12321)
is_palin(1234)
is_palin(madam)
```

## Return keyword

→ The return statement is used to exit a function and go back to the place where it was called.
→ return statement is used to return the value from a function to the function call.
→ We can use only 1 return keyword in function
→ The statement after the return statement will not excute once return statement is executed.
→ Return keyword exit the function and will not come back to the function.
→ Return statement can return single/multiple data.

Note:-

- If we use return keyword inside the function you should use print statement of the function call.
- If we use print statement inside the function no need to use print statement at function call.




With argument without return

```
def add_two (a, b):
    print (a+b)
add_two (10,20)
```

With argument with return

```
def add_two (a,b)
    return (a+b)

res = add_two (10,20)
print (res)
if res % 2 == 0
    print ("even")
else:
    print ("not even")
```

Without argument with return

```
a = int (input ("enter the value"))
b = int (input ("enter the value"))
def add_two ()
    return a+b
print (add_two ())
```

Without argument without return

```
def add_two():
    a=int(input("enter the a value:"))
    b=int(input("enter the b value:"))
    print(a+b)
add_two()
```

Types of Argument

→ positional argument
→ keyword "
→ combination of positional and keyword argument
→ positional only argument
→ keyword " "
→ combination of positional only and keyword only argument
→ Tuple variable len of argument
→ Dictionary " " " "
→ combination of tuple variable len dictionary variable len of argument
→ Default arguments

positional argument

```
def data(P1,P2):
    print(P1,P2,sep=" ")
data(11,12)
```

keyword argument

```
def data(P1,P2):
    print(P1,P2,sep=":")
data(P2=12,P1=11)
```

Combination of positional & keyword Argument

```
def data(P1,P2):
    print(P1,P2,sep=":")
```

position only argument

→ use '/' after all the parameters...

```
def data(P1,P2,/)
    print(P1,P2,sep=":")
data(11,12)
```

keyword only argument [* before all arguments]

```
def data(*,P1,P2)
    print(P1,P2,sep=":")
data(P1=12,P2=13)
```

combination of position only and keyword only.

```
def data(P1,P2,/,*,P3,P4)
    print(P1,P2,P3,P4,sep=":")
data(11,12,P3=14,P4=15)
```

Tuple variable len of argument [to pass n no of positional arg]

```
def data(*args)
    print(args)
#indexing
print(args[0])
#slicing
print(args[1:3])
#loop
total=0
for i in args:
    total+=i
    print(total)
```

Dictionary variable len of Argument

```
def data (**kwargs)          [to pass n no of keywords]
    print (kwargs)
    print (kwargs.values())
    print (kwargs.keys())
    print (kwargs.items())
#dict [key]
    print (kwargs[0])
-# for loop
    total=0
    for v in kwargs.values():
        total+= v
    print (total)
```

Combination of tuple & Dictionary

```
data (a=10, b=20, c=30

def data (*args, **kwargs):
    print (args, kwargs, sep="\n")
data (1,2,3,4, a=22, b=33)
```

Default arguments

```
def add (a=0, b=0, c=0):
    print (a+b+c)

add()      # 0
add (1)    # 1
add (1,2)  # 3
add (1,2,3) # 6
```

Annotation

→ It's like writing a small comment that tells:
- What type of data a variable should hold
- what a function expects and returns

→ use:- for variable and parameter types
→ use:- for return types

ex:-

```
def add_two (a:int, b:int) -> int:
    return a+b
res = add_two (2,6)
print (res)
```

Random . randit

→ random.randit (startnum, endnum)
→ generates the random number.

ex:-

```
import random
print (f"four digit otp: {random.randit (1000, 9999)}")
```

ex:-

```
def guess_random_num (name):
    print (f"welcome to game: {name.upper()}")

    ai_win = 0
    user_win = 0
    for i in range (1,6)
        ai_guess = random.randit (1,10)
        user_guess = int (input ("guess the number:"))
        if user_guess == ai_guess:
            user_win += 1
        print (f"user: {user_guess} ai: {ai_guess} \n
                {name.upper()} won the round")
```

```
else:
    ai_win += 1
    print(f"user: {user_guess} ai: {ai_guess}\n
          {name.upper()} won this round")

if ai_win > user_win:
    print(f"ai is the winner better luck next time:
          {name.capitalize()}")

else:
    print(f"{name.lower()} enjoy ")
guess_random_num(input("enter the player name:"))
```

## Random choice

→ random.choice([ele, ele])
→ It selects the single ele from given collection

Ex

```
import random
print(random.choice(["pizza", "burger", "panipuri",
      "coke", "shawarma"]))
```

## Variable Scope

~~Global variable scope~~

→ Where we can create variable
→ where " " access "
→ Where " " modify "

variable scope

- Global variable scope
- Local variable scope
- Non-local variable scope.

## Global variable scope

variables declared outside any function it called global variable.

scope:
accessible throughout entire program
- outside the function
- inside the function we can access.

condition:
We can use global variable inside the function but we cannot modify it, if we want to modify it we have to use "global" keyword.

Ex:

```
a=10
def srh():
    global a
    a+=1
    print(f"a:{a} inside the function")

srh()
a+=1
print(f"a:{a} outside the function)
```

## Local variable

Variables declared inside a function are called local variables

scope
accessible within the function when they are delared.
- inside the function we can access
- outside the function we cannot access.

Ex:

```
def srh():
    a=10
    print(f"a:{a} inside the function")
```

```
    a+=2
    print(f"a:{a} inside the function after modifying")
srh()
print(a)  # Name error
```

Ex: combination of global and local

```
a=10
def srh():
    global a
    a=30
    a+=1
    print(f"a:{a} inside the function")
srh()
a+=1
print(f"a:{a} outside the function")
```

otp
a: 31
a: 32

## Nonlocal

The nonlocal keyword is used to work with variables inside the nested functions.

→ use the keyword "nonlocal" to create.

Usage:-
When you want to modify a variable from an outer function inside an inner function.

Ex:-

```
a=10
def srh():
    a=100
    print(f"a:{a} inside function")
def csk():
    nonlocal a
    a+=1000
    print(f"a:{a} inside the nested function")
csk()
print(f"a:{a} inside the function after nested")
srh()
a+=1
print(f"a:{a} outside function")
```

## packing and unpacking

→ packing is the phenomenon of grouping the collection of data items into single collection.

→ packing will helps us to keep the data to store, secured, save and easy to maintain

→ Whenever the data transition happen there should be not be data loss. The data packing should happen with the help of a variable length arguments.

→ packing can be done in two ways:

1. Tuple packing (*args)
2. Dictionary packing (**kwargs) (key and value pair).

(**kwargs)

packing

item 1
item 2
item 3
item 4



multiple ele → packing

*args → ()

**kwargs → {}

## Unpacking




multiple ele → extracted

→ Unpacking is a phenominal way of extracting the data from the collection and storing it in the different memories (different arguments)

→ In the sender side (function calling) will be *args or **kwargs at the same time receiver side will be number of arguments created based on the length of the packed collection.

Ex: #str
s="hi"
def str_unpacking (a,b):
    print (a, b, sep="\n")
str_unpacking (*s)

#list
l=["hello", 5, 8.8]
def list_unpacking (a,b,c):
    print (a)
    print (b)
    print (c)
list_unpacking (*l)

#dictionary
d={"z":1, "y":2}
def dictionary_unpacking (z,y):
    print (z,y)

def unpacking (p1,p2):
    pass
unpacking (*collection)

dictionary_unpacking (**d)
#value
print (d[z], d[y])
#pair
print (z, d[z])
print (y, d[y])
dictionary_unpacking (*d)

#unpacking only values
d={"z":1, "y":2}
def dictionary_unpacking (z,y)
    print (z,y)
dictionary_unpacking (**d)

## Recursion

A function calling itself to get the work done is called recursion.

→ A function can call itself (1000 times) not more than that, after that it will break (throw the error (recursion error))

→ To avoid the error we have to give the condition

→ When the condition becomes true it should exit the recursive function.

def fun_name(parameter)
    ...
    fun_name (A)

fun_name (Args)

Ex:
#Recursion limit
import sys
sys.setrecursionlimit (2000)
def nums(n):
    print(n)
    nums (n-1)
nums(3)

#Recursion with 1000 iteration
def nums(n):
    print (n)
    nums (n-1)
nums(3)

WAP to print 3-1 using recursion

```
def nums(n)
    if n==0
        return
    print(n)
    nums(n-1)
nums(3)
```

## Head Recursion :-

executing the va and completing the task




WAP print 1-3 using Recursion

```
def nums(n)
    if n==4
        return
    print(n)
    nums(n+1)
nums(1)
```

## Tail Recursion : completing task and executing the value.

WAP print 1-3 using Recursion

```
def (num(n)
    if n==0
        return
    nums(n-1)
    print(n)
nums(3)
```

## Loops into functions

```
# while loop
s=1
e=10
while s<=e
    print(s)
    s+=1
```

```
#function with while loop
def nums(s,e)
    while s<=e
        print(s, end=" ")
        s+=1
    print() # moves to next line.
nums(1,10)
nums(2,5)
nums(8,20)
```

```
# For loop
Logic of count functions
l= ["rcb", "csk", "mi", "SRH"]
value = input("enter the value:")
count_=0
for ele in l:
    if ele==value
    count_+=1
print(count_)
```

function with FOR LOOP.

```
l=["rcb", "csk", "mi", "SRH"]
def user_count(value, count_=0):
    for ele in l:
        if ele == value:
            count_+=1
    return count_
res = wer_count(input("enter the value:"))
print(res)
```

```
l=["rcb", "csk", "mi", "SRH", "csk"]
def user_count(value, count_=0):
    for ele in l:
        if ele==value:
            count_+=1
    return count_
value = input("enter the value")
print(user_count(value))
```

Create a list of vowels from the A-z range by using while loop.

```
l=[]
def vowel_(s,e)
    while s<=e:
        if chr(s) in "AEIOU":
            print(chr(s))
        s+=1
vowel_(65,90)
```

```
#List of Consonants
l=[]
def
```

# OOPS

**Definition:-**

A language which is based on object is called as object oriented programming language.

**Why oops**

- Readability
- Usability
- Easy to Modify
- Easy to maintain
- Easy to debug
- To store data in a organised way
- To access data in a easy way
- When there is no availability of builtin functions we can create our own user defined functions
- Avoid duplicate attributes
- To store the data in secure.

**class**

class is a container which is the combination of attributes and properties / functionality.

**object**

→ object is the real world entity which is having its own attribute and property, behaviour which describes about it

→ object will be pointing to class

Syntax:-

class className:

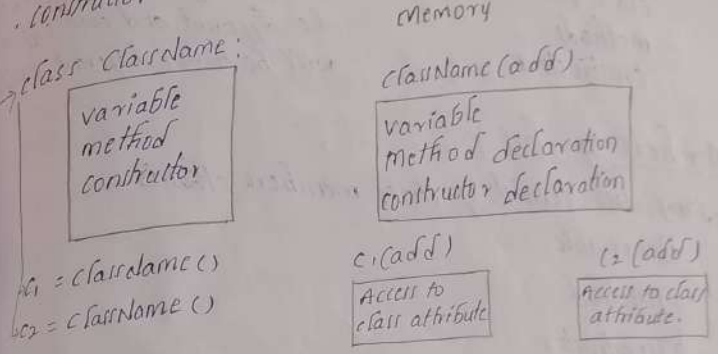


var = ClassName()
var2 = ClassName()

**Members or attributes of the class**

Members of the class is classified into three types

- variable
- Method
- constructor




**class Memory Management of class and object**

~~declared one~~

class Sample:

a=10
b=20

s1 = Sample()
s2 = Sample()

Memory

Sample ( )

| key | value | add |
|---|---|---|
| a | 10 | 1234 |
| b | 20 | 4321 |

* In Ram all the members of class will be stored in dictionary format
* Member name will be "key" and its value will be "value"
* whenever we create A object, for each separate memory allocation will be happen/created and it will be pointing to class
* Each object will be having independent memory allocation.

Syn
classname.__dict__
(or) dir(obj)

s1 ( )

| key | add |
|---|---|
| a | 1234 |
| b | 4321 |

s2 ( )

| key | add |
|---|---|
| a | 1234 |
| b | 4321 |

→ classes are created by keyword class

Syntax to create class

```
class className:
    class variable
    methods
    constructors.
```

Syntax to create object

```
var = ClassName()
```

Address of class and object will be different and each object will be having different address

Members of class [continuation]

→ Any class consist of three members that is

1] variable
2] Method
3] constructor

Variable:-

A variable inside the class is called class variable.

Method:-

Any function inside the class is called as Method.

Constructor:-

We are using a special method that is __init__

Note

→ className should not start with lowercase and no underscore

→ className should start with uppercase character and rest should be in lowercase.

→ And if we having multiple words in className then each word should start with uppercase

Accessing the class variables and Modifying class variable:-

→ If we want to access any members of the class can access in two ways:-

- By using class name
- By using object name

→ Usually we don't use class name to call the variables, Most of the time we use object names.

Syntax

```
classname.var  : → by using class name
objectname.var : → by using object
```

Ex:

```
class Sample:
    a=10
    b=20
s1 = Sample()
s2 = Sample()
print(id(Sample))  # ---352
print(id(s1))      # ---600
print(id(s2))      # ------264
# Accessing the class variable outside the class
#direct
print(a)
#className
print(Sample.a, Sample.b, sep="|")
# obj
print(s1.a, s1.b)  # 10 20
print(s2.a, s2.b)  # 10 20
```

Modification

```
class sample:
    a=10
    b=20
s1=sample()
s2=sample()
#Before modification with className
print(sample.a, s1.a, s2.a, sep="|")
# modification with class Name
sample.a=50
# after modification with className
print(sample.a, s1.a, s2.a, sep="|")
# modification with obj1 Name
s1.a=76
# After modification with obj Name
print(sample.a, s1.a, s2.a, sep="|")
# modification with className after modifying with obj
sample.a=100
# After modification with className
print(sample.a, s1.a, s2.a, sep="|")
```

States / properties / variables

→ These are the data to be stored in the class or object

→ These are classified into Generic states and Specific states.

Generic states (static states):-

→ These are the data that are common for all the objects which are getting created with the help of class.

→ These states are defined in class memory and can be accessed by every object memory using reference object address.

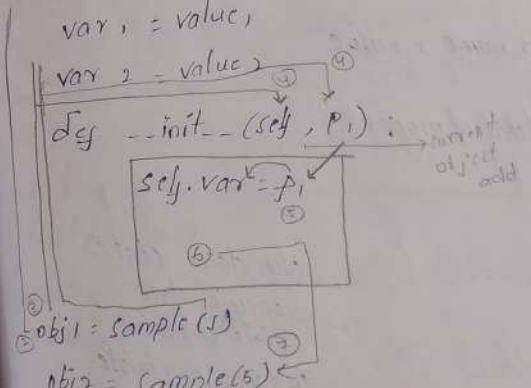
Syntax: to store data in class memory

```
var_nam=value
```

Syn: to fetch data from class memory

```
print(class_name.var_name)
```

→ class sample:



Memory Sample

| key | value |
|---|---|
| var1 | value1 |
| var2 | value2 |
| __init__ | (self, P1) |

obj1 = self

| self.var | 3 |
|---|---|

obj2 = self

| self.var | 5 |
|---|---|

Specific states (non-static states):-

→ These are the data that are specific for each and every object which is getting created in the object memory.

→ These states are not common for the class and other objects.

→ As this data is specific to an object so non static states will be stores inside the object memory.

How to store the data inside an object?

Syn:-

object address.var_name = value

variables/states/properties

Static state
Generic state
class variable

Syntax
var = value

→ stores in class

Non static state
specific state
object variable
Instance variable

Syntax
obj address.var = parameter/value

→ stored in respective object memory.

# static and non-static state

```
class Python:
    # static state
    timing = "3:00 - 5:00 PM"
    batch_code = "PM148"
    sub = "Python"

    def __init__(self, name, age):
        print(id(self), ":self add")
        self.std_name = name
        self.std_age = age
        print(id(self), ":obj address")

s1 = python("ya", 23)
print(id(s1), ":obj address")
```

## Constructor

→ It is one of the special member of the class

→ In python constructor will be defined by the __init__() magic method

→ constructor will get executes whenever we create an object / instance.

→ constructor are used to initialize / initiate instance variable / specific states

→ In simple words to declare instance variable.

Note

To access the same variable in all methods the we declare it as instance variable.

→ The self parameter is the convention
→ when you define a method inside a class you use self as the first parameter.
→ self will hold the address of the current object
→ It represents the instance (object) of the class and allows you to access and manupulate the attributes and methods of that instance.
→ self is a special variable.
→ self is always pointing to current object address/current instance.

Examples of an static state & non static static

```
class Hero:
    name = "Maheshbabu"

def __init__(self, Movie_name, Heroine_name)
    self.Movie_name = Movie_name
    self.Heroine_name = Heroine_name

m1 = Hero(Movie_name: "bahubali", Heroine_name: "Anushka"
m2 = Hero(Movie_name: "salaar", Heroine_name: "Sruthi hasan"
print (Hero.name)
print (m1.name), print (m2.name)
```

To check the data of the class.

syntax:-

```
class_name.__dict__
print (Hero.__dict__)
```

To check the data of the object.

Syntax

```
dir(obj)
print (dir(m1)) #['heroine_name', 'movie_name', 'name']
print (dir(m2)) #['heroine_name', 'movie_name', 'name']
```

Write a program to manage movie information using class and instance Methods.

-> Create class called Movie

-> Inside the class, define the following class variables

-> studio_name

-> total movies

-> Define the following instance variable in the __init__ method.

-> Title, release_year, genre, rating

Create:

create two movie objects with different data.

Sol:-

```
class Movie:
studio_name = "Annapurna_studio"
    total = '0'

    def __init__(self, title, release_year, genre, rating)
        self.movie_title = movie_title
        self.release_year = release_year
        self.genre = genre
        self.rating = rating
m1 = Movie (title: "Fida", release_year: "2018", genre: "love",
                rating: "5")
m2 = Movie (title: "Murari", release_year: "2004", genre: "family"
                rating: "4")
print (Annapurna
```

Write a programe to manage Mobile information using class and instance methods.

```
class Mobile:
    Brand_name(1) = "samsung"
    Brand_name(2) = "Apple"
    total = 0
def __init__(self, model_name, releare_year, price,
             battery_caparity)
    self.model_name = model_name
    self.release_year = release_year
    self.price = price
    self.battery_caparity = battery_caparity

m1 = mobile("samsung", 2022, 12000, 5000mah)
m2 = mobile("vivo", 2023, 14000, 6000mah)
```

Write a program to manage book information using class and instance methods.

-> create class called book
-> Inside the class, define the following class variables
-> library_name
-> total_books (initialize to 0)
-> Define the following instance variable in the __init__ method:
-> Book title, author name, price, pages
-> create two book objects with different data.

```
class Book:
    library_name = "student_library"
def __init__(self, book_title, author_name, price,
             pages):
    self.book_title = book_title
    self.author_name = author_name
    self.price = price
    self.pages = pages
```

## Methods

→ A function which is written inside the class is called as methods.
→ Functions are defined with def keyword.
→ Note: a function which is written outside the class is called as function.

we have three types in methods:-

→ Instance method / object method
→ class method
→ static method.

### Instance method :-

→ The default type of method when you define a method in a class. Takes the instances as first parameter (often named self convention)
→ can access and modify instance-level attributes.
→ Most commonly used for operations that involve the instance's state.

syntax:

```
class class_name
    def methodname(self):
```

why we need instance method

Accessing Instance data: Instance methods can access and modify the data (attributes) of the instance (object) using the self parameter.

object-specific actions:- Since instance methods are associated with instance, they can perform actions that are specific to the state of an object.

code Reusability: Instance methods can be reused across multiple instances of a class. once defined, you can call them on different objects of the same class, making your code more modular and reusable.

### class Method

→ defined using the @classmethod decorator
→ Takes the class itself as the first parameter (often named cls by the convention)
→ can access and modify class-level attributes
→ commonly used for methods that operate on the class rather than on instances.
→ Note :: self wol. pointing to object address
→ Note :: cls will be pointing to classname address
→ syntax:

```
class classname :
    @classmethod
    def methodname(cls, parameter1)
```

Why we need class method

Accessing or modifying class-level attributes

→ class methods have access to the class itself, allowing them to work with class level attributes and perform operations that effect the entire class
→ They are particularly useful when you need to set or modify class variables shared by all instances of the class.
→ class methods do not have access to instance-specific data (eg. instance attributes). This makes them suitable for operations that don't depend on the state of a particular object.

```
class mobile:
    brand_name = "samsung"
    total_mobiles = 0
    def __init__(self, model_name, release_year, price,
                 battery_capacity):
        self.model_name = model_name
        self.release_year = release_year
        self.price = price
        self.battery_capacity = battery_capacity
    @classmethod def brand_details(cls):print(cls.brand_name, cls.total_mobiles)
    def mobile_data(self):
        print(f"MODEL : {self.model_name}\n YEAR : {self.release_year}
              f"PRICE : {self.price}\n BATTERY : {self.battery_capacity}\n

m1 = mobile("S22", 2022, 120000, "3000mah")
m2 = mobile("S25", 2025, 250000, "6000mah")
#Accessing instance method outside the class by using class name
#className.InstanceMethod(obj)
mobile.mobile_data(m1)
mobile.mobile_data(m2)

#objects
#obj.Instance_method()
m1.mobile_data()
m2.mobile_data()

#Accessing class method outside the class by using the
  class name
#className.class_method(obj)
mobile.brand_details()
```

# objects
# obj. class_method()
m1. brand_details()
m2. brand_details()

[code from Prev]

@classmethod
def brand_details(cls):
    print(cls.brand_name, cls.total_mobiles)

def mobile_data(self):

#modify
def modify_price(self):
    self.price = int(input("enter new price:"))
    print("price updated successfully\n")

def modify_release(self):
    self.release = int(input("enter the new release:"))
    print("release year updated successfully")

m1 = mobile("S22", 2022, 120000, "5000mah")
m2 = mobile("S25", 2025, 140000, "7000mah")

m1.modify_price()
m2.modify_price()

m1.modify_release()
m2.modify_relase()

#Increment

```
@classmethod
def increment_mobile_count(cls):
    cls.total_mobiles += 1

def mobile_and_brand(self):
    print(f"mobile name : {self.model_name}\n
            brand name : {self.brand_name}\n")

print(Mobile.total_mobiles)  #0
m1 = Mobile("S22", 2022, 120000, "5000mah")
print(Mobile.total_mobiles)  #1
m2 = mobile("S25", 2025, 140000, "8000mah")
print(Mobile.total_mobiles) #2
```

| Sl.NO | Method Name | CREATING METHOD | ACCESSING OUTSIDE THE CLASS | ACCESSING INSIDE THE CLASS |
|---|---|---|---|---|
| 1 | Instance method | def method_name(self,P1): | 1. ClassName.method(obj.a1)<br>2. obj.method(a1) | 1. cls.method(self.a1)<br>2. self.method(a1) |
| 2 | class method | @classmethod<br>def method_name(cls,P1): | 1. ClassName.method(a1)<br>2. obj.method(a1) | 1. cls.method(a1)<br>2. self.method(a1) |
| 3 | static method | @staticmethod<br>def method_name(P1): | 1. ClassName.method(a1)<br>2. obj.method(a1) | 1. cls.method(a1)<br>2. self.method(a1) |

# static Method

→ Defined using the @staticmethod decorator
→ Takes neither the instance (self) nor the class (cls) as its first parameter
→ Cannot access or modify class or instance variables directly
→ Used for methods that don't depend on instance or class state and are more like utility functions.

Note

Syntax:

```
class classname:
    @staticmethod
    Def methodname(parameter I)
```

Why we need static method

Utility Functions

Sometimes, you may have utility functions that are related to the class but don't depend on any instance variables. These can be implemented as static methods within the class.

Ex:

```
class Employee:
    company = "tcs"
    def __init__(self, name, mobile, age, id):
        self.emp_name = name
        self.emp_mobile = mobile
        self.emp_age = age
        self.emp_id = id
    @staticmethod
    def validate_mobile(user_number):
```

```
st_num = str(user_number)
if len(st_num) == 10 and st_num[0] in ('6','7','8','9'):
    return user_number

else:
    raise Exception("Invalid number")

# if age >= 18:
    return age
else:
    raise Exception("Invalid age")

#e1 = Employee("prabhas", 9908789067, 45, "tcs1")

def access_employee_data(self):
    print(f"EMP NAME : {self.emp_name}\n"
          f"ID = {self.id}\n"
          f"MOBILE = {self.mobile}\n"
          f"AGE = {self.age}\n")

e1 = Employee("prabhas", 9908345601, 45, "tcs1")
e1.access_employee_data()
e2 = Employee("Rana", 6302617891, 47, "tcs2")
e2.access_employee_data()
```

# pillars of OOPs

There are four pillars

* Inheritence
* polymorphism
* Encapsulation
* Abstraction

## Inheritence

→ one of the core concept of oops language

→ The process of inheriting ~~the process~~ from one class to another class is called as inheritence

or

→ The process of inheriting property from parent / super / baseclass to child / sub / derived class is called as Inheritence.

General structure

```
class parent :
    parent class block
class child (parent):
    child class block
```

parent class ?

A parent class (also called a base class or superclass) is the general class that contains common attributes and methods.

child class?

The child class has to access to the attributes and methods of the parent class is called the child class / derived class / subclass.

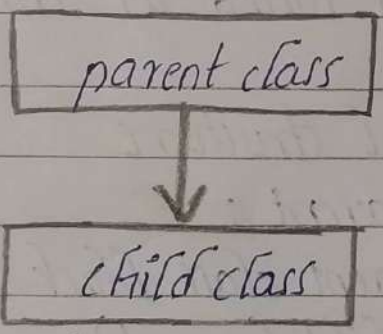
→ It also have its own rules.

→ There are five types in inheritence.

* Single level inheritence
* Multilevel "
* Multiple "
* Hierarchical "
* Hybrid "

Single level Inheritence

→ A child class inherits from only one parent class it is called as single level inheritence




Ex:-

Without inheritence

```
class parent:
    def demo(self):
        print("This is parent class")
P1 = parent()
P1.demo()
```

```
class child:
    def sample(self):
        print("This is child class.")
c1 = child()
c1.sample()
```

With inheritence

```
class Parent:
    def demo(self):
        print("This is parent class")
# P1 = Parent()
# P1.demo()
class child(Parent):
    def sample(self):
        print("This is child class")
c1 = parent()
c1.demo()
c1.sample()
```

O/P: This is parent class
This is child class

Method overriding and Method chaining

Method overriding

When a parent class and child class having same method name after inheritence the child class method will override the properties of parent class method this process is called as method overriding.

## Method chaining

→ To execute the parent class method inside a child class after inheritence is called as method chainning.

→ To achieve method chaining we have two syntaxs

* super().method(A1) (or)
* parentclassName.method(self, A1)

Examples

```
# Method overring
class Parent:
    def demo(self):
        print("this is parent class")
class child(Parent):
    def demo(self):
        print("this is child class")
d1 = child()
d1.demo()
o/p:- This is child class
```

```
class Demo(Example):
    def display(self):
        print("This is child parent class")
class Sample(Demo):
    def display(self):
        Demo().display(self)
        print("This is child class")
```

d1 = Sample()  o/p: this is parent class
d1.display()  this is child class

Method overriding & chaining with parameter

```
class Sample:
    def details (self, name):
        print (f"person : {name}")
class Demo(Sample):
    def details (self, age, name):
        super().details(name)
        print (f"person age : {age}")
d1 = Demo()
d1.details(23, "Indu")
```

Constructor overriding and constructor chaining

Constructor overriding:

When a parent class and child class having constructor after inheritence the child class constructor will overides the properties of parent class constructor. This process is called as constructor overriding

Constructor chaining:

To execute the parent class constructor inside a child class after inheritance

is called as constructor chaining.

→ It can be achieved by using following syntax
* super().__init__()
* parentclassName.__init__(self)

Examples

Constructor overriding

```
class sample:
    def __init__(self):
        print("This is parent class constructor")
class Demo(Sample):
    def __init__(self):
        print("This is child class constructor")
d1 = Demo()
# This is child class constructor
```

Constructor chining

```
class Sample:
    def __init__(self):
        print("This is a parent class")
class Demo(Sample):
    def __init__(self):
        # super().__init__(name)
        Sample().__init__(self, name)
        print("this is child class")
di = Demo()
```

o/p: This is parent class
This is child class

Date
Page

constructor chaining with parameter

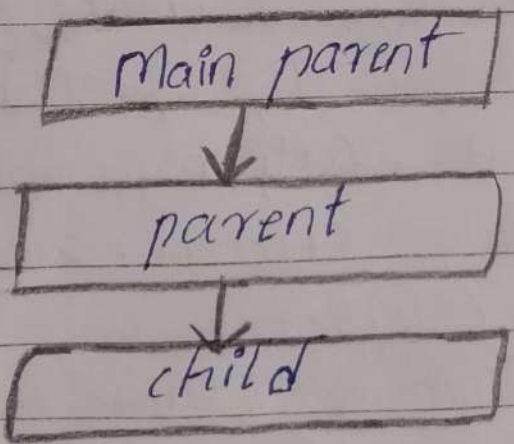
```
class Sample:
    def __init__(self):
        print("This is parent class", name)
class Demo(sample):
    def __init__(self, roll, name):
        #super().__init__(name)
        Sample.__init__(self, name)
        print(f"This is child: {roll}")
di = Demo(23, "Indu")
```

o/p:-

```
# This is parent class: Indu
  This is child: 23
```

## Multilevel Inheritance

child class inheriting properties from parent class and parent class inheriting from its parent class is called as Multilevel Inheritance



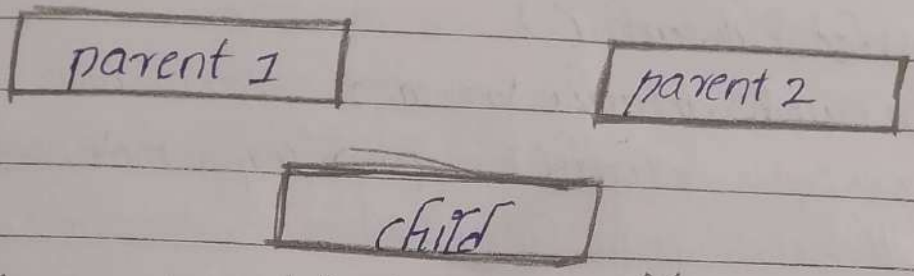
```
class GP:
    g-name = "ballayya"
    def __init__(self, g-property):
        self.g-property = g-property
    def gp-details(self):
        print(f"grandpa name : {self.g-name}\n
                grandpa property : {self.property}")

class Parent(GP):
    p-name = "krishna"
    def __init__(self, P-property, g-property):
        super().__init__(g-property)
        self.p-property = p-property
    def p-details(self):
        print(f"parent name : {self.p-name}\n
                parent property : {self.p-property}\n")

class child(Parent):
    c-name = "Mahesh"
    def __init__(self, c-property, p-property,
                 g-propierty):
        Parent.__init__(self, p-property, g-property)
        self.c-property = c-property
    def c-details(self):
        print(f"childname : {self.c-name}\n
                child property : {self.c-property}\n")
        self.p-details()
        self.gp-details()
c1 = child("100cr", "5000cr", "1000cr")
```

# Multiple Inheritance

A single child class inheritance property from multiple parent class is called as multiple inheritance




super().methodname() ✗

ParentName().methodname() ✓

⇒

```
class Sql:
    def __init__(self, sql_trainer)
    self.sql_trainer = sql_trainer
def sql_details(self)
    print("sql is used to work with database"
          and trainer is:", sql.trainer)

class Web:
    def __init__(self, web_trainer):
        self.web_trainer = web_trainer
    def web_details(self):
        print("Web is the part of F.E and the
                trainer is", web_trainer)

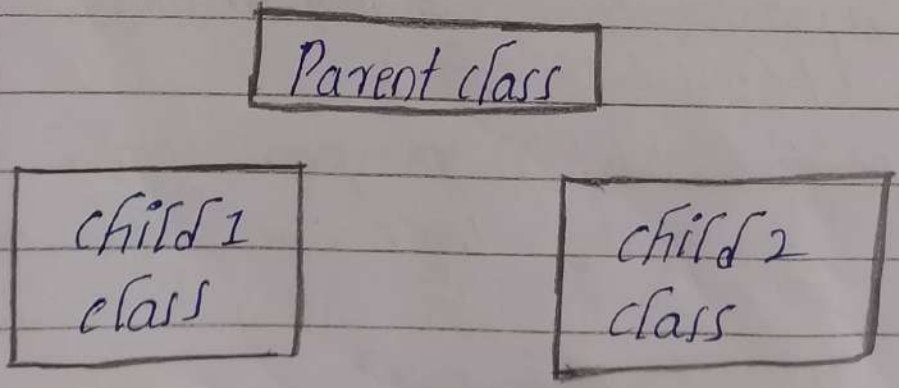
class student(Web, Sql):
    def __init__(self, std_name, web_trainer, sql_trai)
    Web.__init__(self, web_trainer)
    sql.__init__(self, sql_trainer)
```

```
        self.std_name = std_name
    def std_details(self):
        print(f"student_name:{self.name}")
s1 = Student("Indu", "Rahulsir", "Ravitejasir")
s1.std_details()
s1.web_details()
s1.sql_details()   (or) for multiple ways
#.self.web_details()
# self.sql_details()
#s2 = student("Indu", "Gayatrimam", "Yasinsir")
#s2.std_details()

#s3 = student("Indu", "srimatthiman", "Madhulir")
   s3.std_details()
```

## Hierarchical Inheritance

Multiple child class inheriting the property from single parent class is called as hierarchical Inheritance.



```
class Director:
    d_name = "SSRajamouli"
class Bahubali:
    def __init__(self, hero, heroine):
        self.hero = hero
        self.heroine = heroine
    def cast(self):
        print(f"Hero: {self.hero}\nHeroine:
              {self.heroine}\nDirector: {self.d_name}")
b1 = Bahubali("prabhas", "Anushka")
b1.cast()

class Magadheera(Director):
    def __init__(self, hero, heroine):
        self.hero = hero
        self.heroine = heroine
    def cast(self):
        print(f"Hero: {self.hero}\nHeroine: {self.hero
              \nDirector: {self.d_name}")
m1 = Magadheera("Ramcharan", "kajal")
m1.cast()
```

## Hybrid Inheritance

→ combination of two or more inheritance is called as Hybrid Inheritance.

→ Hybrid inheritance is a combination of two or more types of inheritance within a

Single programing language

→ It typically involves a mix of different inheritance types like hierarchical inheritance, multiple inheritance or multilevel inheritance.

→ This allows for greater flexibility in designing class relationships but can also introduce complexities that needs to be managed carefully.

```
class Grandpa:
    def grandpa_property(self):
        print("Grandpa's house")

class Father(Grandpa):
    def father_property(self):
        print("father's car")
        self.grandpa_property()

class Son(Father):
    def son_property(self):
        print("son's bike")
        self.father_property()

class Daughter(Father):
    def daughter_property(self):
```

Singlelevel

Hiera

```
        print("Daughter's scooty")
        self.father_property()
g1 = Grandpa()
g1.grandpa_property()

f1 = father()
f1.father_property()

s1 = Son()
s1.son_property()

d1 = Daughter
d1.daughter.property()
```

mro

→ In python, --mro-- stands for "Method Resolution order"

→ you can access the --mro-- attribute on a class to see the method solution order that python will follow when searching for methods or attributes

→ This can be helpful for understanding the inheritance hierarch and resolving method lookup.

[prev prg from hybrid]

```
print(Grandpa.__mro__)
print(Father.__mro__)
print(son.__mro__)
print(daughter.__mro__)
```

O/P

```
<class '__main__.Grandpa'>, <class 'object'>)
<class '__main__.Father'>, <class '__main__.Grandpa'>, <class 'object'>
<class '__main__.son'>, <class '__main__.Father'>, <class '__main__.Grandpa'>, <class 'object'>
<class '__main__.daughter'>, <class '__main__.Father'>, <class '__main__.Grandpa'>, <class 'object'>
```

storing objec in a list

```
class Emp:
    def __init__(self, name, id):
        self.emp_name = emp_name
        self.id = id
e1 = [Emp("kaushik", 92), Emp("hadra", 126)
```

Composition

Creating the object of a parent class inside a child class is called as composition.

Why composition :-

To access the properties of parent class inside the child class without inheritance is called composition.

```
class Emp:
    def __init__(self, name, id):
        self.emp_name = name
        self.emp_id = id
e1 = Emp("kaushik", 92)
e2 = Emp("badri", 26)
print(e1.emp_name)

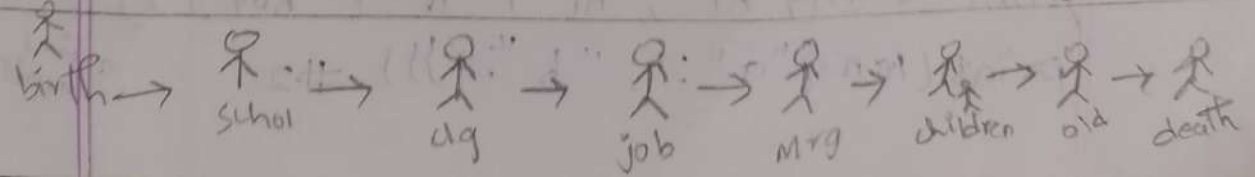
class Sample:
    def __init__(self):
        self.e1 = Emp("kaushik", 92)
    def display_data(self):
        print(self.e1.emp_name)
s1 = Sample()
```

polymorphism

poly → ~~many~~ many

morphism → forms

An object showing different behaviour at different stages of its life cycle is called "polymorphism"



# Duck Typing [no need to specify the type of

```
a=10
print(type(a)) #int
a="hi"
print(type(a))  #str
```

# polymorphism Types

1] Based on operators

+

```
print(10+3)                    # add
print("10"+"3")                # concatination
```

-

```
print(10-3)                    #sub
print({1,2,3}-{3})             #delete
```

*

```
print(2*5)                     #mul
print(2**3)                    # exponent
print("somosse"*3)             # Replication
# packing (*args, **kwargs)
# f-name(*iterable)            # unpacking
```

2] Based on methods

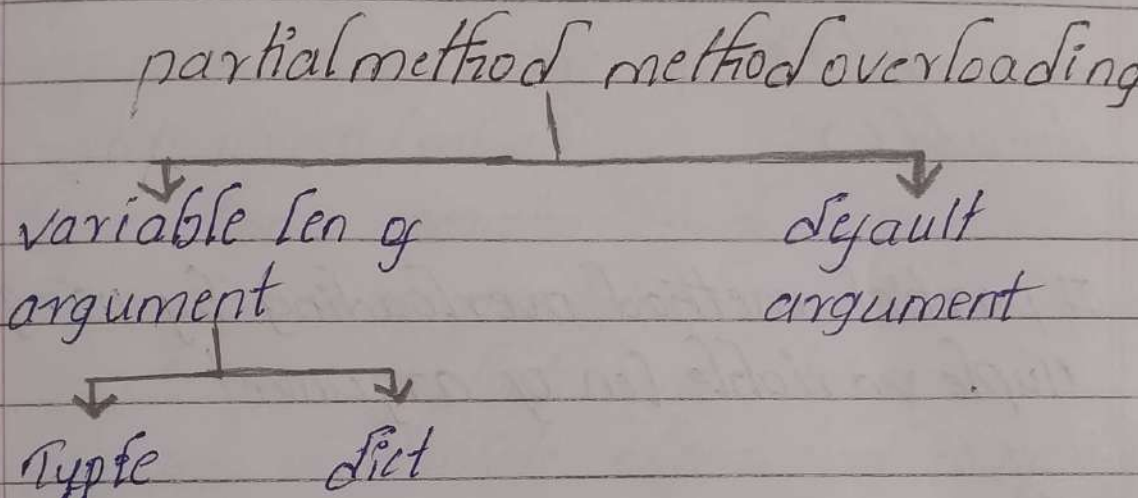
#Len()

```
print(len("hi"))
print([len["hi"]])
print([len["hi"]])
print(len({"hi","hi","hi","hoy"}))  #2
print(len{"a":1,"b":2})  #2
```

avoids duplic

# Method overloading

Single class having same method but different parameters is called as "MOL"
In python method overloading is not possible directly but we can achieve method overloading partially




```
class Demo:
    def add(self,a,b):          #will not execute
        print(a+b)
    def add(self,a,b):          #will execute bcz updated
        print(a+b+c)
d1 = Demo()
d1.add(4,5)                     #error missing argument
d1.add(4,5,6)                   #15
```

```
class Demo:
    def add(self, a=0, b=0, c=0):
        print(a+b+c)
d1 = Demo()
d1.add(4,5,6)
d1.add(4,5)
d1.add(4)
d1.add()
```

# partial method overloading by using Tuple variable len of argument

```
class Sample:
    def add_(self, *args)
        sum_ = 0
        for i in args:
            sum_ += i
        print(sum_)
s1 = Sample()
s1.add_(2,3,4)
s1.add_(4,5,6,7,89,0,6,43,3,2,2,1)
```

# partial method overloading by using Dict variable len of argument.

```
class Sample:
    def add_(self, *kwargs):
        sum_ = 0
        for v in kwargs.values():
```

sum_t = v
print(sum)
s1 = Sample()
s1.add(a=5, v=4, t=6, r=100)

operator overloading.

→ operator overloading means that allows you to define how operators should behave for custom objects/values
→ In python, operators such as +, -, *, /, ==, <, >, etc. are predefined for built-in datatypes like integers, floats and strings
→ you can manipulate objects/values of different types
→ for example, manipulate you can define addition for a custom class to perform concatenation
→ define equality for comparing objects based on specific attributes.
→ These are the magic method which helps us to do operator overloading
→ __add__ : overloading the addition operator +
→ __sub__ : overloading the subtraction operator -
→ __mul__ : overloads the multiplication *
→ __eq__ : overloads the equality operator ==
→ __it__ : overloads the less-than operator <
→ To do operator loading we have to use some special magic method based on functions/methods.

Date
Page

## Encapsulation

→ The process of bundling / binding / wrapping up data / Attributes and methods without within a single unit is called encapsulation

→ When you create a class, it means you are implementing encapsulation. A class is an example of encapsulation as it binds all the data members (instance variables) and methods into single unit.

→ This puts restrictions on accessing variables accidentally or directly and accidentally ~~or directly an~~ modification of methods.

**Note** In python we can't achieve 100% encapsulation

## Access Specifiers

→ There are 3 access specifiers are there in python

- public
- protected
- private

public :- These variables can be accessed anywhere in the class or outside the class & within subcla

| Access Specifiers | static state | Non static state | method | Inside class | outside class | inside subclass |
|---|---|---|---|---|---|---|
| public | var=value | self.var=value | def method(self) | ✓ | ✓ | ✓ |
| protected (for internal use) | _var=value | self._var=value | def _method(self) | ✓ | × | ✓ |
| private (by using name mangling getters) | __var=value | self._var=value | def __method(self) | ✓ | × | × |

we can access outside class

we can access inside subclass & outside class



public access Specifier

```
class sample:
    a=100
    def __init__(self, b)
    self.b=b
    # self.access_data()
    def access_data(self):
        print(f" class var : {self.a}\n obj var : {sub.b}
s1 = Sample(20)

#accessing public dates outside class
print(s1.a)
print (s1.b)
s1.access_data();
#accessing in the subclass
class (sample):
    def access_data
    def access_public_data (self):
        print (self.a, self.b)
        self.access_data()
su1 = sub(6)
su1.access_public_data ()
```

protected

```
class Sample:
    _a = 100
    def __init__(self, b):
        self._b = b
        #self.access_data()
    def _access_data(self):
        print(f"class var: {self._a} \n obj var: {self._b}")
```

#accessing inside subclass

```
class Sub(Sample):
    def _access_protect_data(self):
        print(self._a, self._b)
        self._access_data()
su1 = Sub(6)
su1._access_protected_data()
```

private access specifier

#accessing inside class

```
class Sample:
    __a = 100
    def __init__(self, b):
        self.__b = b
        self.__access_data()
    def __access_data(self):
        print(f"class var: {self.__a} \n obj var: {self.__b}")
s1 = Sample(20)
print(s1._a)  #attribute error
print(s1._b)  #attribute error
s1._access_data()  #attribute error
```

protected Access Specifiers

→ Attributes and methods are accessibly only within the class and its subclasses. They are conventionally prefixed with a single underscore
eg: _protected_attr

→ According to python convention these are the which should not be accessed outside the class

→ These are used only for internal implementation

→ Note:- That nothing technically prevents you from accessing the non-public members directly (protected members) It's just a convention that indicates to other developers that these members are intended for internal use.

Getters and setters

→ In python Getters and setters are not the same as those in other object orientiented programing languages. Basically the main purpose of using getters and setters in object oriented programs is to ensure data encapsulation.

→ private variables in python are not actually hidden fields like in other object oriented languages. Getters and setters in python are often used when:

→ To avoid direct access of a class field i.e. private variables cannot be accessed directly or modified by external user.

```
class Employee:
    def __init__(self, name, age)
        self.emp_name = name
        self.emp_age = age
    def get_age(self)
        return self.__emp_age
    def set_age(self, e_age):
        self.__emp_age = e_age
        print("age updated")
e1 = Employee("prabhat", 16)
e2 = Employee("ramcharan", 41)
#accessing outside directly ×
print(e1.__emp_age)  #Attribute error
print(e2.__emp_age)  #  "  "

#Get
# obj 1
print(e1.get_age())
# obj 2
res = e2.getage()
print(res)
#set
print(f"{e1.get_age()} : before modifying")
e1.set_age(46)
print(f"{e1.get_age()} : after modifying")
```

Chandu Akka 2
May 29, 10:20 AM

# ABSTRACTION

The process of hiding the implementation is called as abstraction.

(or)

Hiding the complexity of features and exposing only the required property to the end users is called as abstraction.

Declaring all the features in abstract class with abstract method decorator.

We should import abc module with ABC class and @ abstractmethod decorator.

abc → modulename
ABC → classname.

## steps to acheive abstraction

1) import module

from abc import ABC, abstractmethod.
↓ module ↓ built-in class ↓ decorator.

2) Inherit the ABC class to parent class
3) use @ abstractmethod before create abstract method
4) use the pass in abstractmethod of parent class
5) implementation will be provided in child class

#steps:

```
from abc import ABC, abstractmethod
class parent(ABC):
    @abstractmethod
    def display(self):
        print("hi this is method")

# obj = parent()
# obj.display() # error.

class Child(parent):
    def display(self):
        print("implementation done")

c1 = child()
c1.display()  # implementation done.
```

ex:2

```
from abc import ABC, abstractmethod:
    class Bank(ABC)
        @abstractmethod
        def check_balance(self):
            pass
    (or)
    @abstractmethod
    def check_balance(self,amount):
        ...
```

declaration part

```
@abstractmethod
def deposit(self, amount):
    ...
@abstractmethod
def withdraw(self, amount):
    pass

class Sbi(Bank):
    def __init__(self, name, balance)
        self.name = name
        self.balance = balance

    def check_balance(self):
        print(f"balance : {self.balance}")

    def deposit(self, amount):
        if amount > 0:
            self.balance += amount
            print(f"rs.{amount} credited successfully and
                your current balance is : {self.balance}\n")

        else:
            print("enter valid amount")

    def withdraw(self, amount):
        if amount <= self.balance:
            self.balance -= amount
            print(f"rs.{amount} debited successfully & your current
                balance is : {self.balance}\n")
```

```
else:
    print ("insufficient balance").
```

$s_1$ = sbi ( "chandu" , 20,000)

$s_2$ = sbi ( "krishna" , 30000)

$s_1$. check_balance( )

$s_2$. deposite (30000)

$s_1$. withdraw (40000) # insufficient balance

$s_1$. check_balance( )

$s_1$. withdraw (5000) # ✓

$S_1$: check_balance()

$S_2$: deposite(30000)

$S_1$: withdraw(40000) # insufficient balance

$S_1$: check_balance()

$S_1$: withdraw(5000) # ✓

29/5/26

## Iterator

Iterator will help us to perform iterations on the Iterable.

Iterable:

It is the collection datatype (or) non-premitive datatype (or) multivalue datatype.

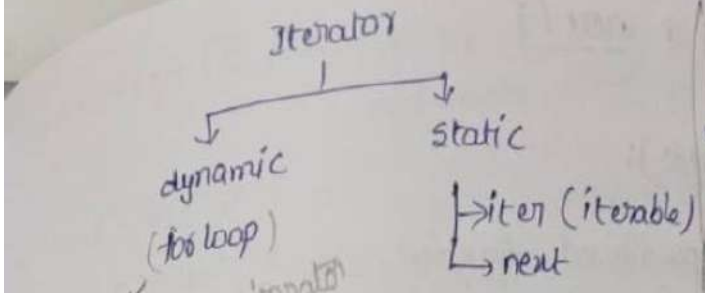
Iteration:

The process of looping is called as Iteration.

Types of Iterators

Iterators are two types. They are

1. Dynamic iterator 2. Static Iterator.




Internally creates a iterator & perform iteration

Eg
s="hel"
v=iter(s)
['h','e','l']
<address stored>
print(next(v))
'h'
'e'
'l'

1. Dynamic iterator:

→ A dynamic iterator in python refers to an iterable object / value that dynamically creates an iterator (or), which helps to perform the iteration.

→ For loop works on the concept of iterator in dynamic nature.

2. Static iterator

In Iterator there are two types of functions which plays major role.

1) iter() 2) next()

INternally Iterators are created with:

magic methods:
1) __iter__
2) __next__

def __init__(self): print("Demo(sample)");

## iter() and next()

→ iter(iterable):

* It is a predifined function.
* The iter() function in python is used to create an iterator object from an iterable.

Syn: Var = iter(iterable)

→ next():

* The next() function in python is used to fetch the next item from an iterator. It takes two arguments.

Syntax: next(Var, [default])

StopIteration error: if we don't have any element to iterate we get this Error.

Ex:

```
1) # List
   L = ["hello", 8, 9.3]
   it = iter()      # print address of list ["hello", 8, 9.3].
   print(it)        # List_iterator obj at 0x000---->
                                          address of list
   print(next(it))  # 'hello
   print(next(it))  # 8
   print(next(it))  # 9.3
# print(next(it))  # StopIteration error
```

print (next (it, "no more ele"))  To avoid the 'Error' we use
                                  second parameter.

1) t = (5, 6, 87, 5)

2) se = {"hello", "hello", 6, 7, 6}.

next() → It fetches the one element at a time from the collection

dict

d = {"a": 1, "b": 2}

it = iter(d)

# keys()

- we don't have second parameter to avoid StopIteration error we use try block i.e, exception handling

```
try:
    print (next (it))   # 'a'
    print (next (it))   # 'b'
    print (next (it))   # StopIteration.

except
    print ("no more ele")
```

# values()

```
d = {'a": 1, "b": 2}

it = iter (d. values( ))   # -- address of [1,2].

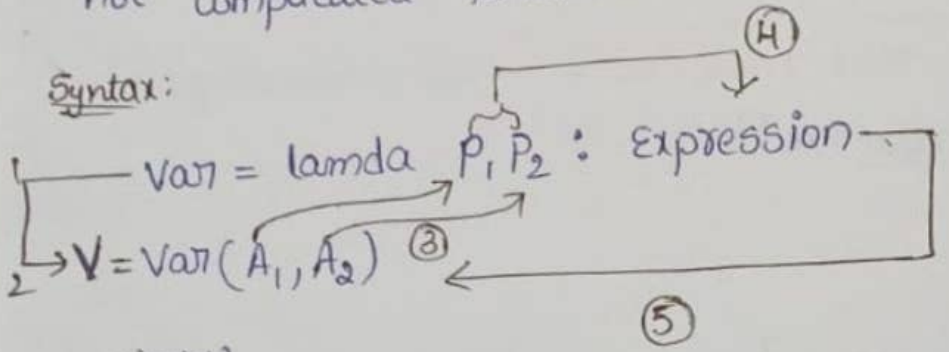
try:
    print (next (it))  # 1
    print (next (it))  # 2
    print (next (it))  # StopIteration
except:
    print (None)
```

# LAMDA FUNCTION

→ lambda → keyword
→ It is anonymous function / lambda expression, it is also called as one line function.
→ we can pass any number arguments.
→ Here lambda will return keyword in lambda expression → (object address)
→ we don't use return keyword in lambda.
→ we don't no name for the lambda, but we can assign it to a variable.
→ lambda functions are many times used as arguments to higher-order functions like map(), filter().
→ lambda is used to write small function in python, not complicated function.

lambda
↓
single line function
↓
Anonymous function.

Syntax:




print(V)

# check even or not by using function.

```
def is_even(n):
    if n%2==0:
        return True
    else:
        return false

res = is_even(4)    # True
print(res)
print(is_even(3))   # false
```

# check even (or) not by using lambda

```
is_even = lambda n : n%2==0
res = is_even(4)
print(res)
```

# check positive (or) not by using lamida

```
is_positive = lambda n : n>0
print(is_positive(4))
print(is_positive(-2))
print(is_positive(0))
```

# get square & cube of the number by using lambda

```
sq_cube = lambda n : f"square:{n**2} cube:{n**3}"
print
res = sq_cube(3)
print(res).
```

```
res=[]
for char, num in zip_longest(s,t, fillvalue=' '):
    res.append(char+str(num))
print(res)
```

## FILE HANDLING

```
import os -> o
```

wap to get the current working directory

```
getcwd()
print(os.getcwd())
```

wap to change

wap to get the current working directory after changing the directory!

```
print(os.getcwd())  # C:\m148
```

wap to create the directory.

```
os.mkdir("sql") -> sql file is created
```

If we want

```
os.mkdir("web")
```

WAP to create the sub directory.

```
os.chdir(r"C:\m148\web")
os.mkdir("html")
```

WAP to remove the directory

```
os.rmdir("css") # filenot found error.
```

If the file is not in folder then we get filenot found error

```
os.rmdir("html")
```

If the file is there in folder that is removed if any content is present in

WAP to get all the files/subdirectory of a directory

```
os.chdir(r"C:\A24")
print(os.listdir()) # ['A26', 'csv files', 'text files']
```

Raw string:

File exist error:

If the file is already present we will get file exist error.

File not found error:

If the file is not present we will get the filenot found error.

File :
open → Two ways to open file
Read/write/append
close

- with out context manager → Var = open("file.ext", "mode")
- with context manager → with open("file.ext", "mode") as var:

"mode" (4) ways

1] without context manager:

file address

```
Var = open("file.ext", "mode")
Var.close()
```

| mode | file not present | file present |
|---|---|---|
| create 'x' | create | file exist error |
| write 'w' | create | Erase all th data & starts from begining. |
| append 'a' | create | adds data at the last |
| read 'r' default mode | File not found error | Read. |

2] with context manager.

```
with open("file.ext", "mode") as var:
```

file addr

It close the file automatically once after exit the block

## Opening File

var.closed → boolean method returns true if the file closed else false.

var.close() → closes the file

→ It is mandatorily used to access the files

```
import os
os.chdir(R"C:\A2H\text files\java")
```

without context manager

```
f_obj = open("rahul_sir.txt")
# print(f_obj)
print(f_obj.closed,": before") # false : before
f_obj.close()
print(f_obj.closed,": after") # True: after
```

with context manager

```
with open("rahul_sir.text","a") as f_obj:
    # print(f_obj)
    print(f_obj.closed,": inside")
print(f_obj.closed,": outside")

# false : inside
# True : outside.
```

Inside context it not give close automatically

→ outside the context it close automatically

returns true if the file i

used to access the files

\java")

false : before

ue: after

f_obj:

inside context it
not close automa

tside the context only

automatically

modes

```
import os
os.chdir(r"c:\m148\sql")
```

# create x

```
with open("joins.text","x") as f_obj:
    print(f_obj)
```

# write

```
with open("subquery.txt","w") as f_obj:
    print(f_obj)
```

# append

```
with open("yasin_sir_assignments.txt","a") as f_obj:
    print(f_obj)
```

# read

```
with open("yasin_sir_assignments.txt","r") as f_obj:
    print(f_obj)
```



Add

TC — [of data]

DI — for loop

[azy — next()

c:\demo\python

Basics.txt — what

Advance.txt — programing lang

oops.txt — token

identified

Datatypes

Typecast

operators

c:\demo\python

'r'

read(): Entire file

readline(): one line at a time

readlines(): [ list of lines ]

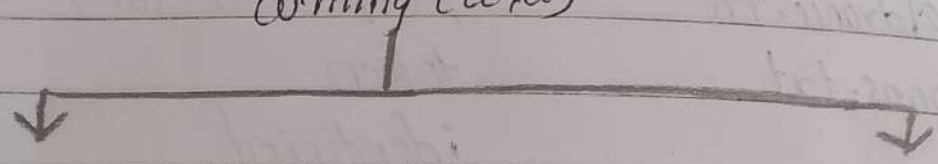
tell() : return index number

seek(index): change the index number

```
os.chdir(r"c:\demo\python")
with open("Basics.txt", "r") as f_obj:
    print(f_obj.read()) #entire file
    print(f_obj.tell()) #150
    f_obj.seek(0)
```

```
print (f_obj.readline())  # 1st line
print (f_obj.readline())  # 2nd  ,,
print (f_obj.tell())  #18
f_obj.seek(0)
f_obj.readlines() ["what\n","prging\n"...]
```

## Writing




writeline

writelines(["line\n","line

```
import os
# os.chdir(r"c:\python")
# os.mkdir(r"text_file")
os.chdir(r"c:\python\text_file")
with open("basic.text","w") as f_obj:
    f_obj.write("what is python ?\n")
    f_obj.writelines(["roadmap\n","tokens\n"])

with open("basic.txt","a") as f_obj:
    f_obj.write("identifier ?\n")
    f_obj.writelines(["keyboard\n","datatypes\n"])
```

# CSV

CSV stands for Coma Separeted value

IPL.CSV

| SL.NO | T.name | star |
|---|---|---|
| 1 | 'CSK' | 'Dhoni' |
| 2 | 'mi' | 'Rohit' |
| 3 | 'SRH' | 'abhi' |
| 4 | 'Rcb' | 'Virat' |

['SL.NO', 'T.Name', 'star']
['1', 'csk', 'Dhoni']
['2', 'mi', 'Rohit']

→ opening/closing/read/writing append are same as textfile
→ all the dates are stored as str datatype
→ we need to import csv module

Reading CSV files

Types of Reading Method.

→ reader()

→ dictreader()

- Here rows of the files will be iterator obj
- which holds each data in the form of python list [[[],[]])

Syntax

- with open("filename.csv", "r") as c_obj:
  csv.reader(c_obj)
- [or]
- csv_obj = open("filename.csv", "r")
- csv.reader(csv_obj)

#reader

```
import os, csv
os.chdir(r"C:\A24\A26\csv_data")
with open("movies.csv", "r") as f_obj:
    r_add = csv.reader(f_obj)
    print(r_add)
    print(list(r_add))
```

DictReader()

- Here Rows of the file will be iterater object
- which holds each data record in the form of dictionary ([{}, {}])

syntax

```
with open("filename.csv", "r") as c_object
        csv.Dictreader(c_object)
```

(or)

```
csv_obj = open("filename.csv", "r")
csv.Dictreader(csv_obj)
```

#DictReader

```
import os, csv
os.chdir(r"C:\A24\A26\csv_data")
with open("ipl.csv", "r") as f_obj:
    dr_add = csv.DictReader(f_obj)
    print(dr_add)
    print(list(dr_add))
```